A COMMUNITY SERVICE PROJECT REPORT

on

SENTINELAUDIT

A Secure Audit-Trail, Asset Versioning and Grounded AI Assistant Platform

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

[BRANCH]

Submitted by

[STUDENT NAME 1] ([REGD. NO.])
[STUDENT NAME 2] ([REGD. NO.])
[STUDENT NAME 3] ([REGD. NO.])
[STUDENT NAME 4] ([REGD. NO.])

Under the esteemed guidance of

[GUIDE NAME]
[Designation]

[COLLEGE LOGO]

DEPARTMENT OF [DEPARTMENT NAME]

SESHADRI RAO GUDLAVALLERU ENGINEERING COLLEGE

(An Autonomous Institute with Permanent Affiliation to JNTUK, Kakinada)

Seshadri Rao Knowledge Village, Gudlavalleru – 521356

[ACADEMIC YEAR, e.g. 20XX – 20XX]

SESHADRI RAO GUDLAVALLERU ENGINEERING COLLEGE

(An Autonomous Institute with Permanent Affiliation to JNTUK, Kakinada)

Seshadri Rao Knowledge Village, Gudlavalleru – 521356

DEPARTMENT OF [DEPARTMENT NAME]

[COLLEGE LOGO]

CERTIFICATE

This is to certify that the Community Service Project report entitled **"SentinelAudit – A Secure Audit-Trail, Asset Versioning and Grounded AI Assistant Platform"** is a bonafide record of work carried out by [STUDENT NAME 1] ([REGD. NO.]), [STUDENT NAME 2] ([REGD. NO.]), [STUDENT NAME 3] ([REGD. NO.]), [STUDENT NAME 4] ([REGD. NO.]) in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in [BRANCH] during the academic year [ACADEMIC YEAR, e.g. 20XX – 20XX].

Project Guide
[GUIDE NAME]
[Designation]

Head of the Department
[HOD NAME]
Professor & HOD

External Examiner
[Signature to be obtained during evaluation]

STUDENT DECLARATION

We hereby declare that the Community Service Project report entitled **"SentinelAudit – A Secure Audit-Trail, Asset Versioning and Grounded AI Assistant Platform"** submitted to the Department of [DEPARTMENT NAME], Seshadri Rao Gudlavalleru Engineering College, Gudlavalleru, is a record of original work done by us under the guidance of [GUIDE NAME]. The technical content of this report describes the software implementation contained in the project repository (github.com/Rahulkagita/SENTINEL-AUDIT-final). This report has not been submitted, either in part or in full, to any other university or institution for the award of any degree or diploma.

Place: Gudlavalleru

Date: [DD/MM/YYYY]

Student Name	**Regd. No.**	**Signature**
[STUDENT NAME 1]	[REGD. NO.]	
[STUDENT NAME 2]	[REGD. NO.]	
[STUDENT NAME 3]	[REGD. NO.]	
[STUDENT NAME 4]	[REGD. NO.]	

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, **[GUIDE NAME]**, [Designation], Department of [DEPARTMENT NAME], for the guidance, encouragement and constructive review provided throughout the Community Service Project.

We are thankful to **[HOD NAME]**, Head of the Department of [DEPARTMENT NAME], for providing the facilities and academic support required to carry out this work.

We extend our thanks to **[PRINCIPAL NAME]**, Principal, Seshadri Rao Gudlavalleru Engineering College, and to the management for creating an environment that encourages project-based and community-oriented learning.

We also thank the Community Service Project coordinator(s) **[CSP COORDINATOR NAME(S)]**, the faculty and technical staff of the department, and our families and friends for their continued support.

[STUDENT NAME 1] ([REGD. NO.])
[STUDENT NAME 2] ([REGD. NO.])
[STUDENT NAME 3] ([REGD. NO.])
[STUDENT NAME 4] ([REGD. NO.])

ABSTRACT

Organisations of every size — small offices, educational institutions, co-operatives and non-profit bodies as well as enterprises — maintain records whose integrity matters: configuration documents, policy texts, data-source descriptions and similar digital assets. When such records change, the questions *who changed what, and when* and *what did it look like before* must be answerable. Manual registers and shared spreadsheets rarely answer them reliably. This project, **SentinelAudit**, is a full-stack web application that records an attributable audit trail for every change to a managed ("protected") asset, keeps restorable version snapshots, and offers a read-only AI assistant that answers questions using only verified documentation and live, allow-listed database reads.

The backend is implemented in **Java 21 with Spring Boot 4.1**, Spring Security and Spring Data JPA over **PostgreSQL**. Authentication is stateless, using HS256-signed **JSON Web Tokens** (JJWT 0.12.6) and BCrypt password hashing; the role is always re-read from the database on each request. Every create, update, soft-delete and restore operation runs in a single transaction that snapshots the prior state into `asset_versions` and writes an `audit_logs` row attributed to the authenticated user's e-mail. The frontend is a **Next.js 13 / React 18 / TypeScript** application using TanStack Query, with pages for a dashboard, protected assets, version history with diff and restore, audit logs, an audit timeline and the assistant.

The assistant implements a complete **retrieval-augmented generation (RAG)** pipeline inside the backend: curated Markdown documents are chunked by heading with overlap, embedded through a pluggable provider (Google Gemini, OpenAI-compatible, or an offline lexical hash provider for tests), stored as JSON vectors in PostgreSQL with SHA-256 change detection, and retrieved by in-process cosine similarity. A bounded tool-calling loop lets the model invoke six read-only data tools, and a citation formatter removes any citation that does not resolve to retrieved content or real tool output. The repository contains 110 backend test methods (JUnit 5, Spring Boot Test, H2) and nine frontend Vitest suites. Known limitations — for example, the absence of tamper-evident hash chaining, per-tenant data isolation and token revocation — are documented explicitly rather than concealed.

Keywords: audit trail, version control, Spring Boot, JWT, Next.js, PostgreSQL, retrieval-augmented generation, embeddings, cosine similarity, citation grounding.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION....1

1.1 Background....1

1.2 About the Project....1

1.3 Motivation and Community Relevance....1

1.4 Scope of the Project....2

1.5 Methodology....2

1.6 Organisation of the Report....3

CHAPTER 2: PROBLEM STATEMENT AND OBJECTIVES....4

2.1 Problem Statement....4

2.2 Objectives....4

2.3 Implementation Status Summary....4

CHAPTER 3: EXISTING SYSTEM AND PROPOSED SYSTEM....6

3.1 Existing System....6

3.2 Drawbacks of the Existing System....6

3.3 Proposed System....6

3.4 Advantages of the Proposed System....7

CHAPTER 4: SOFTWARE AND HARDWARE REQUIREMENTS....8

4.1 Software Requirements....8

4.2 Hardware Requirements....8

4.3 Configuration Requirements....9

4.4 Functional Requirements....9

4.5 Non-Functional Requirements....10

CHAPTER 5: SYSTEM ANALYSIS AND FEASIBILITY STUDY....11

5.1 Requirement Analysis....11

5.2 Use Case Model....11

5.3 Data Flow Analysis....12

5.4 Feasibility Study....13

5.4.1 Technical Feasibility....13

5.4.2 Operational Feasibility....13

5.4.3 Economic Feasibility....13

5.4.4 Schedule Feasibility....13

5.4.5 Legal and Privacy Considerations....13

CHAPTER 6: SYSTEM ARCHITECTURE AND DESIGN....14

6.1 Architectural Overview....14

6.2 Backend Package Structure....15

6.3 Frontend Structure....15

6.4 Request Lifecycle....15

6.5 Authentication and Authorisation Design....16

6.6 Design Patterns and Principles....17
6.7 Deployment Architecture....18
CHAPTER 7: DATABASE DESIGN....19
7.1 Overview....19
7.2 Entity–Relationship Model....19
7.3 Data Dictionary....19
7.4 Aggregate Queries....21
7.5 Integrity Considerations....21
CHAPTER 8: DETAILED MODULE DESCRIPTION....22
8.1 Authentication Module....22
8.2 Protected Asset Management Module....22
8.3 Asset Versioning Module....22
8.4 Audit Logging Module....23
8.5 Dashboard Module....25
8.6 AI Assistant Module....25
8.6.1 Overview....25
8.6.2 Document Ingestion....25
8.6.3 Chunking....25
8.6.4 Embedding Providers....26
8.6.5 Vector Storage and Indexing....26
8.6.6 Retrieval....27
8.6.7 Prompt Construction and Injection Guard....28
8.6.8 Read-only Tool Catalogue....28
8.6.9 Citation Generation and Validation....29
8.6.10 Conversation Bounds and Error Handling....29
8.7 Frontend Modules....29
CHAPTER 9: SYSTEM IMPLEMENTATION....30
9.1 Development Environment....30
9.2 Backend Implementation Highlights....30
9.2.1 Fail-fast JWT key validation....30
9.2.2 Per-request authentication with database-sourced role....30
9.2.3 Transactional update with snapshot and audit....30
9.2.4 History-preserving restore....31
9.2.5 Cosine similarity and citation validation....31
9.3 Frontend Implementation Highlights....31
9.4 Error Handling....32
9.5 Build and Deployment....33
CHAPTER 10: API DOCUMENTATION....34
10.1 Conventions....34
10.2 Endpoint Summary....34
10.3 Authentication Endpoints....35

10.4 Asset Endpoints....35
10.5 Version Endpoints....35
10.6 Audit and Timeline Endpoints....36
10.7 Dashboard Endpoints....36
10.8 Assistant Endpoints....36
CHAPTER 11: TESTING AND VALIDATION....37
11.1 Testing Strategy....37
11.2 Backend Test Suites....37
11.3 Frontend Test Suites....38
11.4 Representative Test Cases....38
11.5 Test Execution Results....38
11.6 Validation Not Performed....39
CHAPTER 12: RESULTS AND DISCUSSION....40
12.1 Delivered Functionality....40
12.2 User Interface....40
12.3 Discussion....43
12.3.1 Atomic, server-attributed audit trail....43
12.3.2 Non-destructive versioning....44
12.3.3 Honest data contracts....44
12.3.4 Grounded assistant design....44
12.3.5 Observations from executing the build....44
CHAPTER 13: LIMITATIONS....45
CHAPTER 14: CONCLUSION AND FUTURE ENHANCEMENTS....47
14.1 Conclusion....47
14.2 Future Enhancements....47
REFERENCES....48
APPENDICES....49
Appendix A – Repository Structure....49
Appendix B – Environment Template (no secrets)....49
Appendix C – Build, Run and Assistant Indexing Procedure....49
Appendix D – Assistant Knowledge Sources....50
Appendix E – Repository Commit History....50
Appendix F – Community Service Activity Records....50

LIST OF FIGURES

Figure 5.1: Use case diagram of SentinelAudit....11
Figure 5.2: Level 0 (context) data-flow diagram....12
Figure 5.3: Level 1 data-flow diagram....12
Figure 6.1: System architecture of SentinelAudit....14
Figure 6.2: Authentication sequence (login and authenticated request)....16
Figure 6.3: Deployment view....18
Figure 7.1: Entity–relationship diagram....19
Figure 8.1: Asset versioning and restore workflow....23
Figure 8.2: Audit-log write and read workflow....24
Figure 8.3: Knowledge ingestion and indexing pipeline....27
Figure 8.4: AI assistant retrieval and answer-generation workflow....28
Figure 12.1: Dashboard with totals, charts and recent activity....41
Figure 12.2: Protected assets list with search, filters and pagination....41
Figure 12.3: Version history of an asset....42
Figure 12.4: Audit log table....42
Figure 12.5: Audit timeline....43
Figure 12.6: Assistant page in the not-configured state....43

LIST OF TABLES

Table 2.1: Implementation status of principal capabilities....5
Table 3.1: Comparison of existing practices and SentinelAudit....7
Table 4.1: Software requirements (as declared in the repository)....8
Table 4.2: Indicative hardware requirements....8
Table 4.3: Principal environment variables....9
Table 5.1: Principal use case descriptions....11
Table 6.1: Backend packages (com.sentinelaudit.backend)....15
Table 6.2: Frontend source organisation (frontend/src)....15
Table 6.3: Endpoint access rules (SecurityConfig)....17
Table 7.1: users....19
Table 7.2: protected_assets....20
Table 7.3: asset_versions....20
Table 7.4: audit_logs....20
Table 7.5: assistant_knowledge_chunks....20
Table 8.1: Audit events produced by the system....24
Table 8.2: Embedding provider implementations....26
Table 8.3: Assistant tools (AssistantToolCatalog)....28
Table 8.4: Frontend pages....29
Table 9.1: Exception-to-status mapping....32
Table 10.1: REST endpoint summary....34
Table 11.1: Backend test classes and number of @Test methods (from source)....37
Table 11.2: Frontend Vitest suites (counts as reported by the test run)....38
Table 11.3: Representative test cases and expected behaviour....38
Table 11.4: Observed test and build results....39
Table 12.1: Requirement traceability....40
Table A.1: Documents indexed by the assistant....50
Table A.2: Commit history of the repository (git log)....50

LIST OF ABBREVIATIONS

API	Application Programming Interface
BCrypt	Blowfish-based adaptive password hashing function
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSRF	Cross-Site Request Forgery
DFD	Data-Flow Diagram
DTO	Data Transfer Object
ER	Entity–Relationship
HS256	HMAC with SHA-256
JPA	Jakarta Persistence API
JPQL	Jakarta Persistence Query Language
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SHA-256	Secure Hash Algorithm, 256-bit
UI	User Interface

CHAPTER 1
INTRODUCTION

1.1 Background

Almost every organisation keeps digital records whose correctness must be defensible: data-source registrations, configuration descriptions, policy texts, inventories and similar artefacts. In a regulated enterprise these records are subject to formal audit; in a small office, a school, a co-operative society or a non-governmental organisation they are typically kept in spreadsheets or shared folders. In both settings the same questions recur whenever something goes wrong: *Who changed this record? When? What was its previous content? Can the earlier state be recovered?* An **audit trail** — a chronological, attributable record of operations — together with **versioning** — retention of prior states — is the standard engineering answer to these questions.

Over the last few years a second need has emerged. Users increasingly expect to *ask* a system about its own state in natural language (for example, “Which assets were deleted this week and by whom?”). Large language models (LLMs) make such interfaces possible, but an LLM answering from its own parametric memory can produce confident, fabricated statements. In an audit context a fabricated answer is worse than no answer. Retrieval-augmented generation (RAG) [10] mitigates this by grounding the model in retrieved evidence, and explicit citation validation makes the grounding verifiable.

1.2 About the Project

SentinelAudit is a full-stack web application that combines these two ideas. It manages a catalogue of *protected assets* — named records with a type, description, optional content payload, status, department, owner and risk level. Every create, update, soft-delete and restore operation on an asset is attributed to the authenticated user and recorded as an audit event, and every modification first captures a snapshot of the previous state so that any earlier version can be inspected, compared and restored. A dashboard summarises totals and trends, an audit-log view and a timeline expose the event history, and a read-only **SentinelAudit Assistant** answers questions using curated product documentation plus a fixed set of read-only database tools, returning only citations it can verify.

The system is implemented as two deployable units held in a single repository (`Rahulkagita/SENTINEL-AUDIT-final`): a **Spring Boot 4.1 / Java 21** REST backend (`backend/`) backed by **PostgreSQL**, and a **Next.js 13 / React 18 / TypeScript** single-page frontend (`frontend/`). The backend ships with a multi-stage Dockerfile; the frontend includes Netlify build configuration.

1.3 Motivation and Community Relevance

The Community Service Project programme encourages students to apply engineering skills to problems that matter beyond the classroom. Record-keeping

integrity is such a problem: small institutions and community organisations commonly lack affordable tools for change accountability, while commercial governance suites are expensive and complex. SentinelAudit is built entirely on open-source components, runs on a single modest server, and requires no paid service unless the optional AI assistant is enabled. Its design goal is therefore *accessible accountability*: a deployable, understandable audit trail with honest documentation of what it does and does not guarantee.

This report documents the software artefact. Field activities associated with the Community Service Project — such as visits, surveys or user interactions — are not recorded in the repository and are therefore **not** described here; Appendix F provides a placeholder for the student to attach the officially certified activity records.

1.4 Scope of the Project

- User self-registration (configurable), login and current-user lookup using stateless JWT authentication.
- CRUD management of protected assets with server-side search, filtering and zero-based pagination; deletion is a soft delete.
- Automatic version snapshots before each update, delete and restore; read-only version history and restore to any earlier version.
- An audit log of CREATE, UPDATE, DELETE and RESTORE events with actor, asset reference, timestamp and remark; paginated listing, filtering and a per-asset timeline.
- Dashboard aggregates: totals, audit actions by type, assets by type, version growth per day and recent activity.
- A read-only RAG assistant with pluggable chat and embedding providers, idempotent knowledge indexing, bounded tool calling and verified citations.
- Containerised backend build and environment-variable-driven configuration with no secrets in source control.

Out of scope (and explicitly not implemented): cryptographic tamper evidence, field-level diffs stored on the server, IP/user-agent capture, anomaly detection, multi-tenant isolation, administrative role management and token revocation. These are discussed in Chapter 13.

1.5 Methodology

The work followed an iterative, increment-based approach visible in the repository history (Appendix E): an initial combined frontend/backend commit was followed by build fixes, correction of the asset tab and audit timeline, removal of mock data so that every screen is backed by the real API, addition of the assistant, and preparation for deployment. Each increment was accompanied by automated tests (JUnit 5 / Spring Boot Test on the backend, Vitest on the frontend) and by a written engineering record (`PROTOTYPE_AUDIT.md`) that lists what was changed, what was verified and what remains unverified.

1.6 Organisation of the Report

Chapter 2 states the problem and objectives. Chapter 3 contrasts existing approaches with the proposed system. Chapter 4 lists software and hardware requirements. Chapter 5 presents system analysis, use cases, data-flow diagrams and feasibility. Chapter 6 describes the architecture and the authentication design. Chapter 7 documents the database. Chapter 8 describes each module in detail, including the AI assistant pipeline. Chapter 9 covers implementation, error handling and deployment. Chapter 10 is the API reference. Chapter 11 covers testing, Chapter 12 results and discussion, Chapter 13 limitations and Chapter 14 conclusions and future work, followed by references and appendices.

CHAPTER 2
PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

Records that describe an organisation's data assets change frequently and are edited by several people. Without system support, (i) changes are not reliably attributed to an individual, (ii) previous states are lost when a record is overwritten, (iii) deletions are irreversible and leave no trace, and (iv) answering questions about the history of a record requires manual inspection. Introducing an AI question-answering interface on top of such data adds a further risk: (v) the model may produce plausible but unsupported statements, which is unacceptable in an accountability context.

The problem addressed by this project is therefore: **to design and implement a secure web application that records an attributable, queryable audit trail and restorable version history for managed assets, and that offers a natural-language assistant whose answers are grounded only in verifiable documentation and live read-only data, while being transparent about the guarantees it does not provide.**

2.2 Objectives

1. Provide stateless authentication with hashed passwords and signed, expiring access tokens, and protect every non-public endpoint.
2. Allow authenticated users to create, view, search, filter, update and soft-delete protected assets with server-side pagination.
3. Capture a snapshot of an asset's previous state before every modification, expose the version history, and support restoring any earlier version without losing history.
4. Record an audit event, attributed to the authenticated user, for every state-changing operation, atomically with the change itself.
5. Expose audit logs and a per-asset timeline with filtering, and summarise activity on a dashboard.
6. Build a read-only assistant that retrieves relevant documentation by vector similarity, can query live data only through an allow-list of validated tools, and returns only citations that resolve to real evidence.
7. Keep all secrets out of the repository, fail fast on insecure configuration, and never leak internal error details to clients.
8. Verify behaviour with automated unit, integration and contract tests.

2.3 Implementation Status Summary

Table 2.1 classifies the principal capabilities according to the state observed in the source code. "Partial" denotes a capability that exists but with documented restrictions.

Table 2.1: Implementation status of principal capabilities

Capability	Status	Evidence / remarks
JWT login, registration, /auth/me	Implemented	AuthController, UserService, JwtService, JwtAuthFilter
Registration on/off switch	Implemented	FeatureFlags + REGISTRATION_ENABLED; 403 when disabled
Logout	Partial	Stateless no-op endpoint; token remains valid until expiry (no revocation list)
Role-based authorisation	Partial	Role stored and loaded into authorities (default USER) but no endpoint restricts by role
Asset CRUD with filters & pagination	Implemented	ProtectedAssetController / Service using JPA Specifications
Soft delete	Implemented	deleted=true, status=DELETED; history retained
Version snapshots & restore	Implemented	AssetVersionService.recordSnapshot / restoreVersion
Unique (assetId, version) constraint	Partial	Enforced by service-level upsert; no database unique constraint
Audit logging & timeline	Implemented	AuditLog entity, AuditLogService, AuditLogController
Field-level diff stored server-side	Not implemented	Diff is computed in the UI from version snapshots
Hash-chain tamper evidence, IP/user-agent capture	Not implemented	DTO fields deliberately returned empty
Anomaly detection / risk scoring	Not implemented (deferred)	Documented decision in PROTOTYPE_AUDIT.md §4
Dashboard aggregates	Implemented	DashboardController / DashboardService
RAG assistant (chat, status, reindex)	Implemented	assistant package; disabled (503) without a provider key
Gemini and OpenAI-compatible providers	Implemented	GeminiLlmClient, OpenAiLlmClient, matching embedding providers
pgvector / external vector database	Planned	In-process cosine over JSON vectors is used instead
Streaming answers, persisted conversations	Not implemented	Answers are returned whole; history is client-held
Multi-tenant isolation	Not implemented	All authenticated users read all records

CHAPTER 3
EXISTING SYSTEM AND PROPOSED SYSTEM

3.1 Existing System

In the absence of a dedicated tool, change accountability for organisational records is commonly handled in one of the following ways. These are general practices, described here to motivate the design; no survey data is claimed.

- **Manual registers and spreadsheets.** A shared workbook lists assets; edits overwrite cells. Some tools retain file-level revision history, but changes are not tied to individual records, and restoring one record without disturbing others is cumbersome.
- **Database-level logging.** Triggers or database audit features capture row changes. They require database administration skills, are tied to a specific vendor, and are rarely exposed to end users in a readable form.
- **General-purpose version control.** Tools such as Git provide excellent history but are designed for files and developers, not for non-technical users editing structured records through a form.
- **Commercial governance suites.** Enterprise data-governance and GRC platforms provide audit trails and catalogues but carry licensing cost and operational complexity beyond the reach of small institutions.
- **Unconstrained AI chatbots.** Generic chat assistants can be pointed at exported data, but they answer from model memory, do not cite verifiable sources, and may expose data to third parties without control.

3.2 Drawbacks of the Existing System

1. Changes are not reliably attributed to an authenticated individual.
2. Previous states of an individual record are lost or difficult to recover.
3. Deletions are destructive and leave no trace.
4. History is not queryable by operation, actor, asset or text.
5. AI answers are not grounded or verifiable, and may be fabricated.
6. Secrets and configuration are often embedded in files or scripts.

3.3 Proposed System

SentinelAudit replaces these practices with a dedicated, browser-based application. Users authenticate with an e-mail address and password and receive a signed access token. Every asset operation is executed by a transactional service that (a) derives the actor from the server-side security context — never from the request body — (b) snapshots the prior state into a version table, (c) applies the change, and (d) records an audit event. Because all four steps share one transaction, a failed operation leaves neither a partial change nor an orphaned audit row; this property is asserted by the test `failedMutationCreatesNoAuditEvent`.

Deletion is implemented as a soft delete, so history remains readable afterwards. Restoring a version never rewrites history: the current state is first snapshotted, then the chosen version's content is applied as a new, higher version number, and a RESTORE event is written. The assistant is strictly read-only and evidence-bound: it may cite only documentation chunks that were retrieved for the current question, or assets and events that a tool actually returned; any other citation marker is removed before the answer reaches the user.

3.4 Advantages of the Proposed System

- Attributable audit trail written atomically with each change.
- Non-destructive versioning with one-click restore and a side-by-side diff view in the UI.
- Server-side filtering and pagination that remain correct beyond the first page of results.
- Stateless JWT security with fail-fast secret validation and uniform, non-leaking error responses.
- Optional grounded assistant with provider choice (Gemini, OpenAI-compatible, offline local) and verified citations.
- Open-source stack, single-container backend, and configuration entirely through environment variables.
- Honest presentation: fields the backend cannot populate (hash, IP address, user agent) are returned empty rather than invented.

Table 3.1: Comparison of existing practices and SentinelAudit

Criterion	**Spreadsheet / manual**	**DB triggers**	**SentinelAudit**
Per-record attribution	Weak	Database user only	Authenticated application user (e-mail)
Per-record version restore	Manual	Manual SQL	Built-in, history-preserving
Deletion recoverability	No	Depends	Soft delete; history retained
End-user query of history	Manual	Requires SQL	Filtered logs, timeline, assistant
Grounded natural-language Q&A	No	No	Yes (optional, read-only, cited)
Tamper evidence	No	Vendor-dependent	No (documented limitation)

CHAPTER 4
SOFTWARE AND HARDWARE REQUIREMENTS

4.1 Software Requirements

Table 4.1 lists the software components with the versions declared in `backend/build.gradle`, `backend/Dockerfile` and `frontend/package.json`.

Table 4.1: Software requirements (as declared in the repository)

Layer	Component	Version / notes
Language (backend)	Java (OpenJDK / Eclipse Temurin)	21 (Gradle toolchain; Docker image eclipse-temurin:21)
Framework	Spring Boot	4.1.0 (plugin), dependency-management 1.1.7
Starters	data-jpa, security, validation, webmvc	Managed by Spring Boot BOM
Security token library	JJWT (api, impl, jackson)	0.12.6
Boilerplate reduction	Lombok	Managed version (compile-only)
Database	PostgreSQL + JDBC driver	Runtime driver org.postgresql:postgresql
Test database	H2 (in-memory)	testRuntimeOnly
Build tool	Gradle wrapper	Included (gradlew / gradlew.bat)
Testing (backend)	JUnit 5, Spring Boot Test, Spring Security Test	Managed versions
Language (frontend)	TypeScript	5.2.2
UI framework	React / Next.js	18.2.0 / 13.5.1
Data fetching	@tanstack/react-query	^5.101.2
Styling & components	Tailwind CSS, Radix UI, lucide-react	3.3.3; Radix 1.x/2.x
Forms & validation	react-hook-form, zod	^7.53.0, ^3.23.8
Charts	recharts	^2.12.7
Testing (frontend)	Vitest	^3.2.7
Runtime (frontend)	Node.js	Compatible with Next.js 13 (Node 18+ recommended)
Container	Docker	Multi-stage Dockerfile for backend
Optional AI providers	Google Gemini API / OpenAI-compatible API	Defaults: gemini-3.8-flash, gemini-embedding-001@768; gpt-4o-mini, text-embedding-3-small
Browser	Any modern evergreen browser	Chrome, Edge, Firefox, Safari

4.2 Hardware Requirements

The repository does not prescribe hardware and no performance benchmarking was performed. Table 4.2 therefore gives *indicative* minimums derived from the requirements of the runtime components (JVM, Node.js and PostgreSQL) for development and small-scale deployment; they are not measured values.

Table 4.2: Indicative hardware requirements

Resource	Development machine	Small deployment (single server)
Processor	64-bit dual-core, 2 GHz or better	2 vCPU
Memory (RAM)	8 GB (IDE, JVM, Node.js, PostgreSQL concurrently)	2–4 GB
Storage	5 GB free (JDK, Gradle cache, node_modules)	10 GB+ depending on data volume
Network	Internet access for dependency download	HTTPS; outbound access only if the assistant is enabled
Client	Any device with a modern browser	—

4.3 Configuration Requirements

All runtime configuration is supplied through environment variables (`application.properties` uses `${VAR:default}` placeholders). Table 4.3 lists the principal variables; secret values are never committed and are shown here only as descriptions.

Table 4.3: Principal environment variables

Variable	Required	Purpose / default
JWT_SECRET	Yes	Base64 string decoding to ≥ 32 bytes; startup fails otherwise
JWT_EXPIRATION	No	Token lifetime in ms; default 86 400 000 (24 h)
DATABASE_URL / DATABASE_USERNAME	Yes	JDBC URL (default localhost/sentinelaudit) and user (default postgres)
DATABASE_PASSWORD	Yes	No default — empty value fails on connect
PORT	No	HTTP port; default 8080
CORS_ALLOWED_ORIGINS	No	Comma-separated browser origins allowed to call the API
REGISTRATION_ENABLED	No	true by default; false makes /api/auth/register return 403
GEMINI_API_KEY / GEMINI_MODEL	No	Enables Gemini chat and embeddings
ASSISTANT_LLM_* / ASSISTANT_EMBEDDING_*	No	Provider, base URL, key, model, timeouts, dimensions
ASSISTANT_RETRIEVAL_TOP_K / MIN_SCORE / MAX_CHUNKS	No	6 / 0.12 / 5000
ASSISTANT_INDEXING_ALLOW_API / ASSISTANT_INDEX_ON_STARTUP	No	Both false by default
API_PROXY_TARGET (frontend)	No	Target for /api/* rewrite; default http://localhost:8080
NEXT_PUBLIC_API_BASE_URL (frontend)	No	Overrides same-origin /api base URL

4.4 Functional Requirements

1. FR1 – The system shall register users (when enabled) and authenticate them by e-mail and password.
2. FR2 – The system shall reject requests to protected endpoints that lack a valid, unexpired token with HTTP 401.
3. FR3 – The system shall allow creation, retrieval, update and soft deletion of protected assets.
4. FR4 – The system shall support search, department, status and risk filters with zero-based pagination and reject unsupported parameters.
5. FR5 – The system shall snapshot an asset’s state before each update, delete and restore, and expose the version history.
6. FR6 – The system shall restore any earlier version as a new version without deleting history.
7. FR7 – The system shall write an audit event for every CREATE, UPDATE, DELETE and RESTORE, attributed to the authenticated user.
8. FR8 – The system shall list and filter audit logs and timeline events by operation, free text and asset.
9. FR9 – The system shall provide dashboard totals and aggregates.

10. FR10 – The system shall answer natural-language questions using retrieved documentation and read-only tools, returning verified sources.

4.5 Non-Functional Requirements

- **Security:** BCrypt password hashing; HS256 JWT with ≥ 256-bit key; stateless sessions; CORS allow-list; no secrets in source; generic error messages.
- **Integrity:** asset change, snapshot and audit row committed in one transaction.
- **Honesty:** no fabricated values in responses (hashes, IPs, severities) and no fabricated assistant answers on provider failure.
- **Maintainability:** layered packages, DTO mapping, provider interfaces enabling substitution and testing with fakes.
- **Portability:** containerised backend; environment-driven configuration; H2 for tests.
- **Usability:** responsive UI with light/dark themes, loading skeletons, empty states and toast notifications.

CHAPTER 5
SYSTEM ANALYSIS AND FEASIBILITY STUDY

5.1 Requirement Analysis

The requirements in Chapter 4 were analysed to identify actors, use cases and data stores. Two human actors interact with the system: a **Visitor**, who may only register (when enabled) and log in, and an **Authenticated User**, who may use every other function. Because the backend assigns the role `USER` to every self-registered account and no endpoint is restricted by role, all authenticated users currently have identical capabilities. Two external system actors are involved: the **PostgreSQL** database and, optionally, an **LLM / embedding provider**.

5.2 Use Case Model

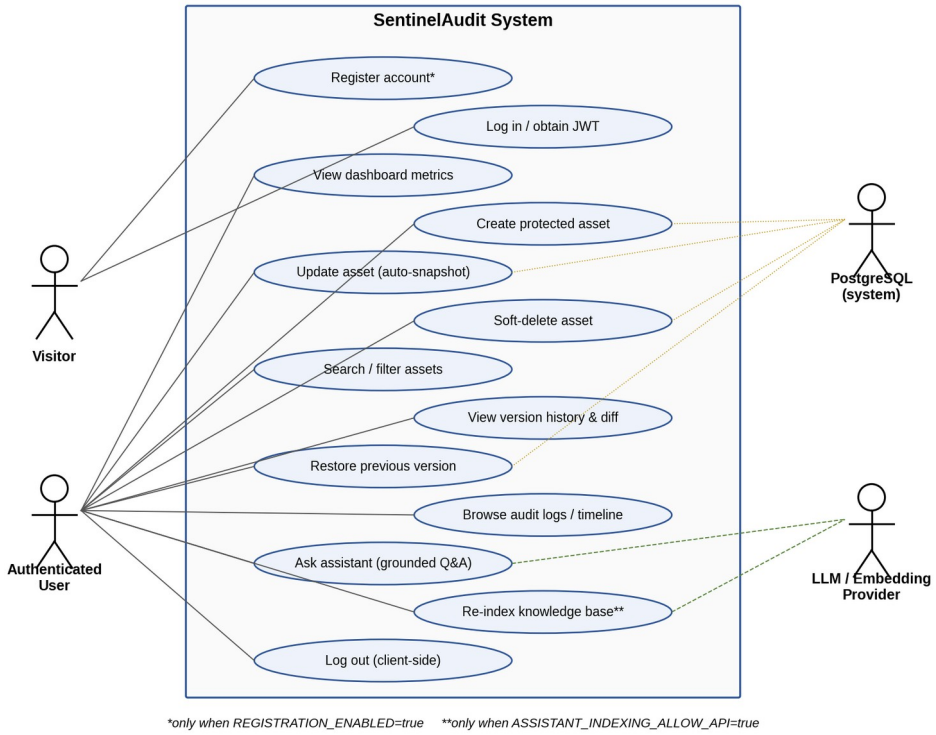


Figure 5.1: Use case diagram of SentinelAudit

Table 5.1: Principal use case descriptions

Use case	Actor	Pre-condition	Main flow / outcome
Log in	Visitor	Account exists	Submit e-mail + password → JWT and user profile returned; stored in browser
Create asset	User	Valid JWT	Submit name and type (required) and metadata → asset v1 saved; CREATE event
Update asset	User	Asset exists, not deleted	Snapshot current version → apply supplied fields → version+1; UPDATE event
Soft-delete asset	User	Asset exists, not deleted	Snapshot → deleted=true, status=DELETED; DELETE event; later reads return 404
View history & diff	User	Asset row exists	List versions newest first; compare snapshots in UI
Restore version	User	Asset not deleted; version	Snapshot current → copy vN fields → new

Use case	Actor	Pre-condition	Main flow / outcome
		exists	version; RESTORE event
Browse audit logs / timeline	User	Valid JWT	Paginated, filtered events newest first
Ask assistant	User	Provider configured	Question → retrieval + tools → answer with verified sources (503 if unconfigured)
Re-index knowledge	User	ASSISTANT_INDEXING_ALLOW_API=true	Chunks embedded/skipped/removed; report returned (403 if disabled)

5.3 Data Flow Analysis

Figure 5.2 shows the context-level (level 0) data-flow diagram, treating SentinelAudit as a single process exchanging data with its external entities. Figure 5.3 decomposes the system into six processes and five data stores corresponding to the database tables described in Chapter 7.

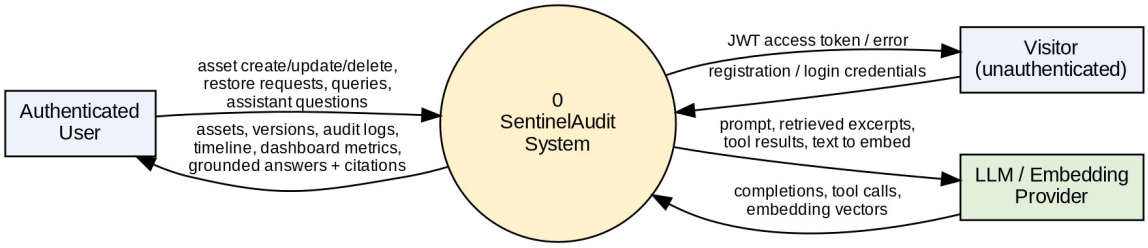


Figure 5.2: Level 0 (context) data-flow diagram

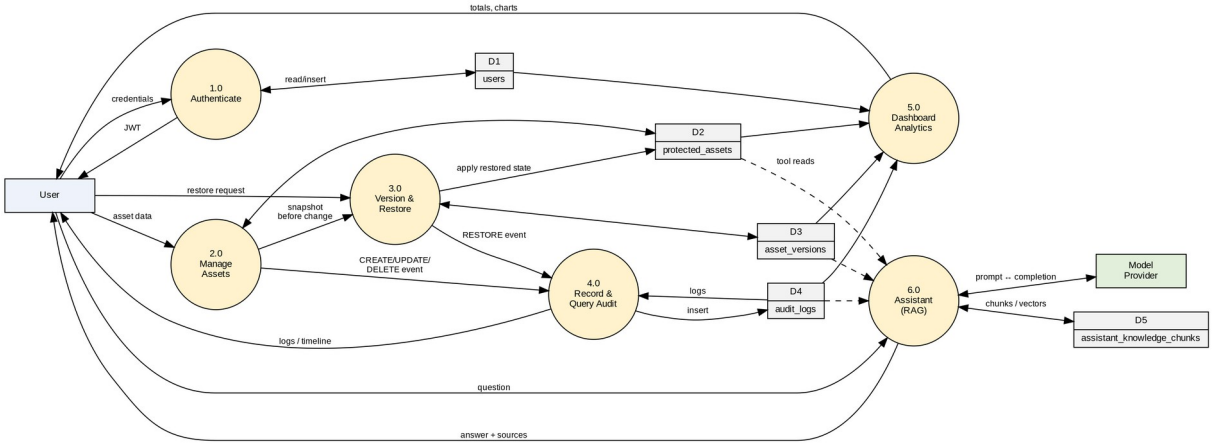


Figure 5.3: Level 1 data-flow diagram

The level 1 diagram highlights two design properties. First, processes 2.0 (Manage Assets) and 3.0 (Version & Restore) never write to the audit store directly through the client — the audit event is a side effect produced by the server within the same transaction. Second, process 6.0 (Assistant) reads the asset, version and audit stores only through service methods exposed as read-only tools, and writes only to its own knowledge-chunk store during indexing.

5.4 Feasibility Study

5.4.1 Technical Feasibility

All components are mature, widely documented open-source technologies (Spring Boot, Spring Security, Hibernate, PostgreSQL, React, Next.js). The RAG pipeline was deliberately implemented without additional infrastructure: vectors are stored as JSON text in an ordinary table and similarity is computed in the JVM, avoiding a dependency on the `pgvector` extension, which cannot be assumed to be installed on a hosting provider. The approach is exact and adequate for a curated corpus of hundreds of chunks. The project is therefore technically feasible with standard student hardware.

5.4.2 Operational Feasibility

Users interact through a conventional web interface with sidebar navigation (Dashboard, Protected Assets, Audit Logs, Audit Timeline, Settings) and a floating assistant. Operations that change data provide immediate feedback, and caches for assets, timelines and audit logs are invalidated after each mutation so that views do not show stale data. Administrators configure the system solely through environment variables, and the application fails fast with explicit messages when mandatory settings such as `JWT_SECRET` are missing or weak.

5.4.3 Economic Feasibility

No licensed software is required. Development uses free tools, and deployment requires one server or container host and a PostgreSQL instance, for which free or low-cost tiers exist. The only potentially recurring cost is the optional model provider, which is billed by the provider according to usage; the assistant is disabled by default and the rest of the system functions without it. Re-indexing skips unchanged chunks (SHA-256 comparison), which avoids repeated embedding charges.

5.4.4 Schedule Feasibility

The repository history (Appendix E) records nine commits between 18 and 20 October 2026, covering the integrated frontend/backend, defect fixes, removal of mock data, the assistant subsystem and deployment preparation. Work performed before the initial commit is not visible in the history and is not estimated here.

5.4.5 Legal and Privacy Considerations

When the assistant is enabled, the question, retrieved documentation excerpts and tool results are sent to the configured provider endpoint; `README-ENVIRONMENT.md` explicitly calls for a privacy review before enabling it in production. API keys are read only on the server and are never exposed through `NEXT_PUBLIC_*` variables, logged, or returned by the status endpoint.

CHAPTER 6
SYSTEM ARCHITECTURE AND DESIGN

6.1 Architectural Overview

SentinelAudit follows a classic **three-tier architecture** (Figure 6.1). The presentation tier is a Next.js application rendered in the browser; the application tier is a Spring Boot REST service; the data tier is a PostgreSQL database. An optional fourth element, the external model provider, is used only by the assistant subsystem and only from the server side.

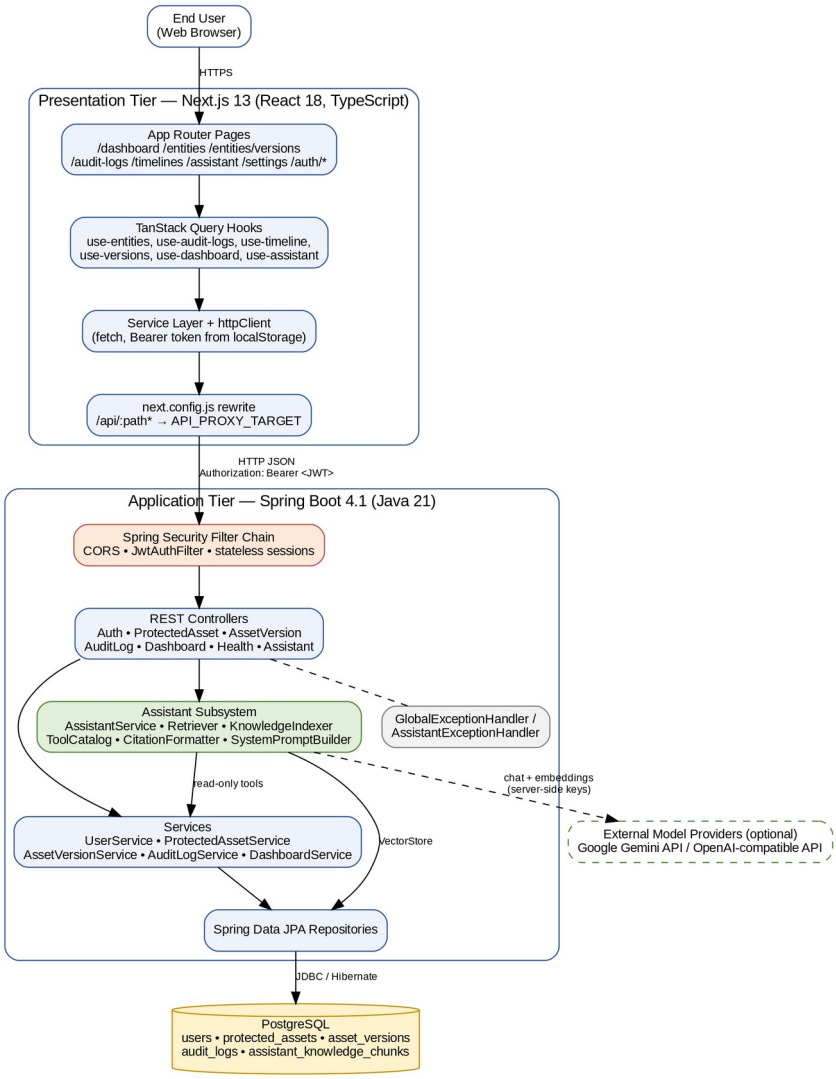


Figure 6.1: System architecture of SentinelAudit

Within the backend a **layered architecture** is used: the security filter chain authenticates the request; controllers validate input and map DTOs; services implement business rules and transactions; Spring Data JPA repositories perform persistence. Cross-cutting error translation is centralised in two `@RestControllerAdvice` classes. The assistant subsystem is a self-contained package that depends on the existing services through narrow, read-only calls.

The browser never talks to the backend origin by default. The frontend calls same-origin `/api/...`, and `next.config.js` rewrites `/api/:path*` to `API_PROXY_TARGET` (default `http://localhost:8080`). This avoids CORS configuration in development; for split-origin deployments `NEXT_PUBLIC_API_BASE_URL` can point directly at the backend, which then relies on its explicit CORS allow-list.

6.2 Backend Package Structure

Table 6.1: Backend packages (com.sentinelaudit.backend)

Package	Responsibility	Key classes
(root)	Application entry point	BackendApplication
security	JWT issue/verify, request filter, filter-chain and CORS	JwtService, JwtAuthFilter, SecurityConfig
config	Password encoder bean, feature flags	SecurityBeansConfig, FeatureFlags
controller	REST endpoints for auth, assets, versions, audit, health	AuthController, ProtectedAssetController, AssetVersionController, AuditLogController, HealthController
service	Business logic and transactions	UserService, ProtectedAssetService, AssetVersionService, AuditLogService
dashboard	Aggregate statistics	DashboardController, DashboardService, *Response
entity / repository	JPA entities and Spring Data repositories	User, ProtectedAsset, AssetVersion, AuditLog; *Repository
dto / response	Request/response contracts	AssetResponseDto, AuditLogResponseDto, TimelineResponseDto, VersionResponseDto, ApiResponse, PaginatedResponse
exception	Typed exceptions and global handler	BadRequest, Unauthorized, Forbidden, NotFound, Conflict, RegistrationDisabled; GlobalExceptionHandler
assistant.*	RAG assistant: knowledge, provider, retrieval, vector, prompt, tools, citation, dto	AssistantService, KnowledgeIndexer, Retriever, AssistantToolCatalog, CitationFormatter

6.3 Frontend Structure

Table 6.2: Frontend source organisation (frontend/src)

Folder	Contents
app/	Next.js App Router pages: dashboard, entities, entities/versions, audit-logs, timelines, assistant, settings, auth/login, auth/register, not-found
features/	Feature components: assistant (chat, markdown, floating widget, citation and error helpers), audit-logs (table, detail drawer), auth (layout, protected route, session loader, validation), dashboard (overview, charts, activity feed), entities (table), timelines (timeline view)
hooks/	TanStack Query hooks: use-auth, use-entities, use-versions, use-audit-logs, use-timeline, use-dashboard, use-assistant, use-toast
services/	Typed API clients: http-client, auth, assets, version, audit, dashboard, assistant
providers/	AppProviders (query client, theme) and AuthProvider (session state)
components/	Layout (app shell, sidebar, topbar, page header), common widgets (diff viewer, stat cards, badges, skeletons) and Radix-based UI primitives
lib/, types/	Environment, constants, pagination translation, formatting utilities; shared TypeScript types

6.4 Request Lifecycle

A typical authenticated request proceeds as follows: (1) a page component calls a hook such as `useEntities(params)`; (2) the hook calls the service, which calls `httpClient.get('/assets', {params})`; (3) `httpClient` reads the token from `localStorage` key `sentinel_audit_token` and adds `Authorization: Bearer`

<token>; (4) the Next.js rewrite forwards the request; (5) on the backend, the CORS filter and `JwtAuthFilter` run; (6) Spring Security authorises the request (`anyRequest().authenticated()`); (7) the controller validates parameters and calls the service; (8) the response DTO is serialised as JSON. Any non-2xx status is converted on the client into an `HttpClientError` carrying the status code and server message.

6.5 Authentication and Authorisation Design

Authentication is **stateless**: no HTTP session is created (`SessionCreationPolicy.STATELESS`) and CSRF protection is disabled because no cookie-based credentials are used. Figure 6.2 shows the login and subsequent authenticated-request sequence.

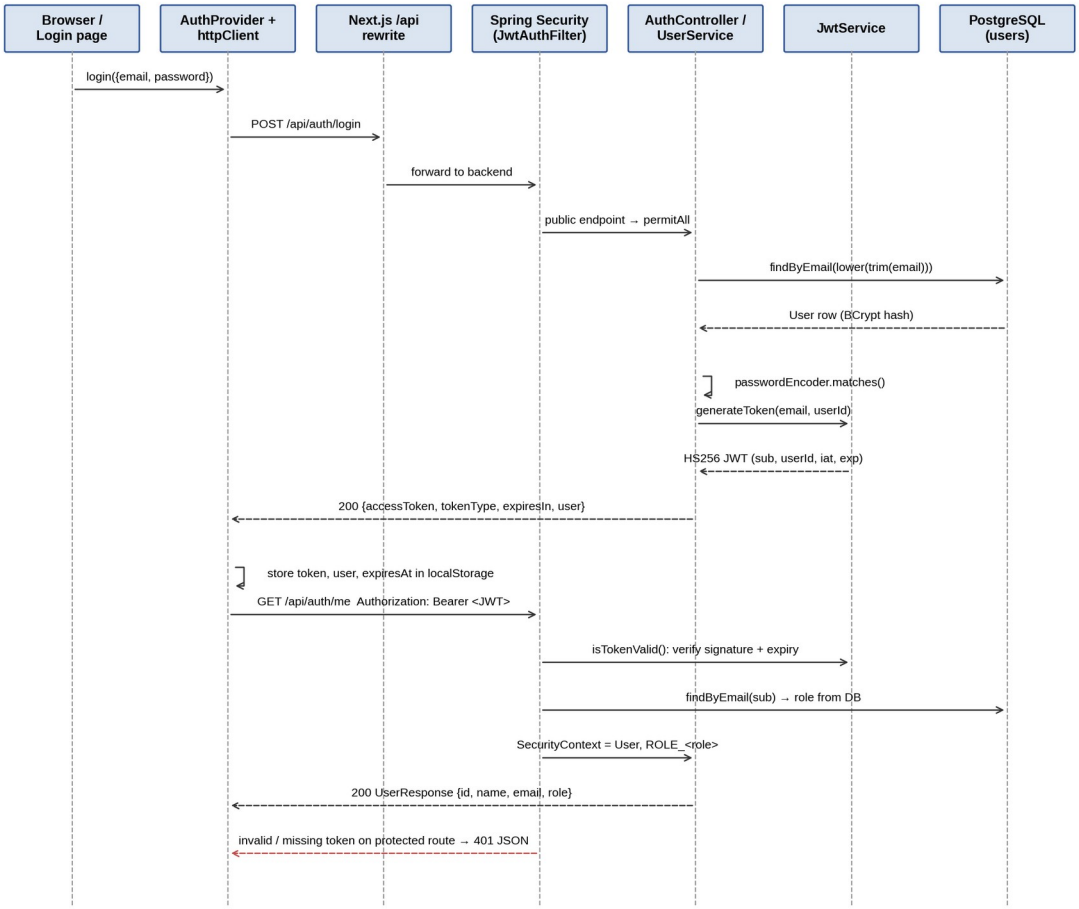


Figure 6.2: Authentication sequence (login and authenticated request)

- **Registration** (`POST /api/auth/register`) is first gated by `FeatureFlags.isRegistrationEnabled()`. The e-mail is trimmed and lower-cased, duplicates yield 409, the password is hashed with `BCryptPasswordEncoder`, and the role is always `USER`. A token is issued immediately (HTTP 201).
- **Login** (`POST /api/auth/login`) looks up the normalised e-mail and compares the BCrypt hash. Unknown e-mail and wrong password produce the same 401 response, preventing account enumeration.

- **Token issue**: `JwtService.generateToken` creates an HS256 token with subject = e-mail, claim `userId`, `iat` and `exp` (default 24 h). The response contains `accessToken`, `tokenType` = Bearer, `expiresIn` (seconds) and the user profile; no refresh token is issued.
- **Key management**: at startup `JwtService.init()` rejects a missing, non-Base64, or shorter-than-32-byte secret, so a weak configuration aborts boot instead of failing at first login.
- **Request authentication**: `JwtAuthFilter` (a `OncePerRequestFilter` placed before `UsernamePasswordAuthenticationFilter`) verifies signature and expiry, loads the user by e-mail and sets an authentication whose principal is the `User` entity and whose authority is `ROLE_<role>` read **from the database**, not from the token.
- **Authorisation**: OPTIONS pre-flight, `/api/auth/login`, `/api/auth/register`, `/api/auth/config` and `/api/health` are public; everything else requires authentication. Unauthenticated access returns a JSON 401 and access-denied returns a JSON 403.
- **Client session**: `AuthProvider` stores the token, user and computed expiry in `localStorage`, drops expired sessions on reload, and `ProtectedRoute` redirects unauthenticated users to `/auth/login?redirect=<path>`.

Table 6.3: Endpoint access rules (SecurityConfig)

Pattern	Method	Access
/**	OPTIONS	Public (CORS pre-flight)
/api/auth/login, /api/auth/register, /api/auth/config	Any	Public
/api/health	Any	Public
All other paths	Any	Authenticated (valid JWT)

6.6 Design Patterns and Principles

- **Layered architecture / separation of concerns** — controller, service and repository layers.
- **Data Transfer Object** — entities are never serialised directly by asset, audit and version endpoints; DTOs adapt them to the frontend contract (e.g. status mapping ACTIVE → connected, version → "v3").
- **Repository and Specification** — Spring Data repositories, with `JpaSpecificationExecutor` building dynamic filter predicates.
- **Strategy** — `EmbeddingProvider` and `LlmClient` interfaces with Gemini, OpenAI-compatible and local implementations chosen at startup by `AssistantConfig`.
- **Template / bounded loop** — the assistant turn is a fixed pipeline (retrieve → ground → tool loop → validate citations).
- **Centralised exception translation** — `@RestControllerAdvice` maps typed exceptions to status codes with generic messages.
- **Fail-fast configuration** and **least privilege** — secret validation at boot; default role USER; read-only assistant tools.

6.7 Deployment Architecture

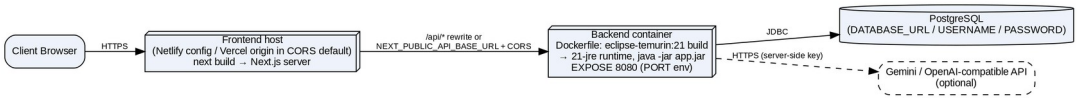


Figure 6.3: Deployment view

The backend Dockerfile uses a two-stage build: an `eclipse-temurin:21` stage runs `./gradlew clean bootJar`, and an `eclipse-temurin:21-jre` stage runs `java -jar app.jar`, exposing port 8080. The frontend contains a `netlify.toml` (build command `npx next build`, `@netlify/plugin-nextjs`), and the default CORS allow-list includes `http://localhost:3000` and a Vercel origin. The repository does not contain CI pipeline definitions or infrastructure-as-code; actual hosting details are therefore not documented here.

CHAPTER 7
DATABASE DESIGN

7.1 Overview

The system uses PostgreSQL in production and an in-memory H2 database (PostgreSQL compatibility mode in one profile) for automated tests. The schema is generated from JPA entities by Hibernate with `spring.jpa.hibernate.ddl-auto=update`, which only adds missing tables and columns and never drops data; no separate migration tool (e.g. Flyway or Liquibase) is used. Five tables exist: four for the core domain and one for the assistant's knowledge index.

7.2 Entity–Relationship Model

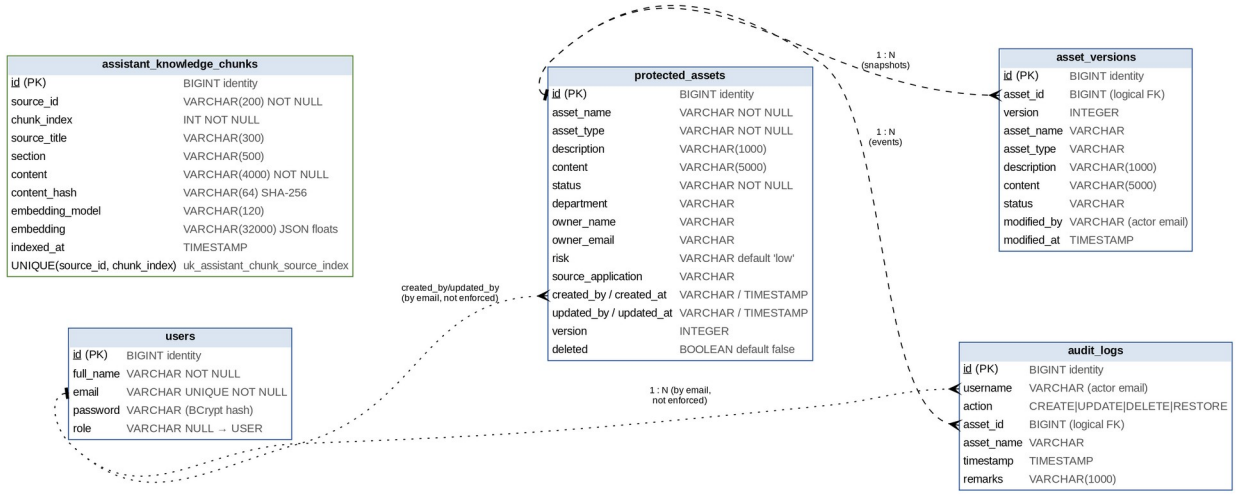


Figure 7.1: Entity–relationship diagram

Relationships between tables are **logical**, not enforced by foreign-key constraints: `asset_versions.asset_id` and `audit_logs.asset_id` hold the identifier of a `protected_assets` row as a plain `BIGINT`, and actor columns (`created_by`, `updated_by`, `modified_by`, `username`) store the user's e-mail as text. This design keeps audit and version rows readable even if the referenced row changes, and is consistent with soft deletion (asset rows are never physically removed by the application). The trade-off is that referential integrity is maintained by the service layer rather than the database.

7.3 Data Dictionary

Table 7.1: users

Column	Type	Constraints	Description
id	BIGINT	PK, identity	Surrogate key
full_name	VARCHAR(255)	NOT NULL	Display name
email	VARCHAR(255)	UNIQUE, NOT NULL	Login identifier, stored lower-case
password	VARCHAR(255)	NOT NULL	BCrypt hash (never returned by any API)
role	VARCHAR(255)	Nullable	USER for self-registered accounts; null treated as USER

Table 7.2: protected_assets

Column	Type	Constraints	Description
id	BIGINT	PK, identity	Asset identifier
asset_name / asset_type	VARCHAR(255)	NOT NULL	Display name and category
description	VARCHAR(1000)		Optional free-text metadata
content	VARCHAR(5000)		Optional payload (API only; not collected by UI form)
status	VARCHAR(255)	NOT NULL	e.g. ACTIVE (default), DELETED; mapped to UI values
department, owner_name, owner_email	VARCHAR(255)		Ownership metadata
risk	VARCHAR(255)	default 'low'	low / medium / high
source_application	VARCHAR(255)	default 'SentinelAudit'	Origin label
created_by, created_at	VARCHAR, TIMESTAMP		Actor e-mail and time of creation
updated_by, updated_at	VARCHAR, TIMESTAMP		Actor e-mail and time of last change
version	INTEGER		Current version number (starts at 1)
deleted	BOOLEAN	default false	Soft-delete flag

Table 7.3: asset_versions

Column	Type	Constraints	Description
id	BIGINT	PK, identity	Snapshot identifier
asset_id	BIGINT	Logical FK	Asset to which the snapshot belongs
version	INTEGER	Unique per asset (service-enforced)	Version number captured
asset_name, asset_type, status	VARCHAR(255)		Captured field values
description / content	VARCHAR(1000) / (5000)		Captured field values
modified_by, modified_at	VARCHAR, TIMESTAMP		Actor e-mail and capture time

Table 7.4: audit_logs

Column	Type	Constraints	Description
id	BIGINT	PK, identity	Event identifier
username	VARCHAR(255)		Actor e-mail, or SYSTEM when no user principal
action	VARCHAR(255)		CREATE, UPDATE, DELETE or RESTORE
asset_id, asset_name	BIGINT, VARCHAR		Affected asset reference and name at the time
timestamp	TIMESTAMP		Server time of the event
remarks	VARCHAR(1000)		Short remark, e.g. “Restored version v2”

Table 7.5: assistant_knowledge_chunks

Column	Type	Constraints	Description
id	BIGINT	PK, identity	Row identifier
source_id	VARCHAR(200)	NOT NULL; UNIQUE with chunk_index	Document id, e.g. product-overview
chunk_index	INT	NOT NULL	Zero-based position in document
source_title / section	VARCHAR(300) / (500)		First H1 heading; heading path (A > B)
content	VARCHAR(4000)	NOT NULL	Chunk text

Column	Type	Constraints	Description
content_hash	VARCHAR(64)	NOT NULL	SHA-256 hex of chunk content
embedding_model	VARCHAR(120)		Model id that produced the vector
embedding	VARCHAR(32000)		Vector encoded as JSON float array
indexed_at	TIMESTAMP		Time of last embedding

Column lengths not specified in the entities (marked VARCHAR(255)) follow Hibernate's default string length. The unique constraint `uk_assistant_chunk_source_index (source_id, chunk_index)` is what makes re-indexing idempotent.

7.4 Aggregate Queries

Dashboard statistics are computed with JPQL aggregate queries declared on the repositories:

```
-- AuditLogRepository.getAuditActionCounts()
SELECT a.action, COUNT(a) FROM AuditLog a GROUP BY a.action

-- ProtectedAssetRepository.getAssetTypeCounts()  (excludes soft-deleted rows)
SELECT p.assetType, COUNT(p) FROM ProtectedAsset p
WHERE p.deleted IS NULL OR p.deleted = false GROUP BY p.assetType

-- AssetVersionRepository.getVersionGrowth()
SELECT DATE(a.modifiedAt), COUNT(a) FROM AssetVersion a
GROUP BY DATE(a.modifiedAt) ORDER BY DATE(a.modifiedAt)
```

Note that `DashboardSummaryResponse.totalAssets` uses `count()` and therefore includes soft-deleted assets, whereas the assets-by-type aggregate excludes them; the assistant's `dashboard_summary` tool states this explicitly in its result.

7.5 Integrity Considerations

- Asset change, snapshot and audit row are written inside one `@Transactional` service method.
- Version numbers are kept unique per asset by `recordSnapshot`, which upserts on `(assetId, version)`; there is no database unique constraint, so duplicates created before this logic existed are not cleaned up.
- Legacy rows with `deleted = NULL` or `role = NULL` are handled explicitly (treated as not deleted / USER).
- Audit rows are append-only by API: no endpoint updates or deletes audit logs, although nothing at the database level prevents a database administrator from doing so.

CHAPTER 8
DETAILED MODULE DESCRIPTION

8.1 Authentication Module

Implemented by `AuthController`, `UserService`, `JwtService`, `JwtAuthFilter`, `SecurityConfig`, `SecurityBeansConfig` and `FeatureFlags` on the backend, and by `AuthenticationService`, `AuthProvider`, `ProtectedRoute` and the login/register pages on the frontend. Section 6.5 describes the flow. Additional behaviours: `GET /api/auth/config` returns only `{registrationEnabled}` so the UI can hide the sign-up form, and the frontend fails open on this call (the server still enforces the rule); `RegisterRequest` validates a non-blank name, a valid e-mail and a 6–100 character password, mirroring the client-side zod schema; `POST /api/auth/logout` returns 204 and the client discards its token.

8.2 Protected Asset Management Module

`ProtectedAssetController` (`/api/assets`) and `ProtectedAssetService` implement asset CRUD. On creation the service sets status `ACTIVE`, risk `low` and source application `SentinelAudit` when not supplied, version 1, `deleted=false`, and records `createdBy/updatedBy` as the current actor. Listing is fully server-side: the controller rejects any query parameter other than `page, pageSize, search, department, status, risk`, requires `page ≥ 0` and `1 ≤ pageSize ≤ 200`, limits text filters to 200 characters and validates enumerations. The service builds a JPA `Specification` that:

- always excludes soft-deleted rows (`deleted IS NULL OR deleted = false`);
- matches `search` case-insensitively against name, type, department, owner name and owner e-mail, or exactly against the numeric id;
- matches `department` as a case-insensitive substring;
- maps UI status values to stored aliases (connected → connected/active, disconnected → disconnected/inactive);
- matches `risk` exactly (case-insensitive), and sorts by id descending.

Updates are partial: blank `assetName`/`assetType` are ignored; `description`/`content` are changed only when the field is present (an explicit empty string clears it); other metadata are overwritten only when non-blank. `AssetResponseDto` adapts the entity to the frontend's `Entity` contract, e.g. `changeCount = version − 1` and `version = "v" + version`.

8.3 Asset Versioning Module

`AssetVersionService` and `AssetVersionController` (`/api/versions`) implement history and restore (Figure 8.1). `recordSnapshot` writes the asset's name, type, description, content and status, with actor and time, under a given version number, overwriting any existing row for that `(assetId, version)` pair. Snapshots are taken

before every update and delete and twice during a restore (before and after). History reads require only that the asset row exists, so history stays readable after soft deletion; restore is refused for deleted assets.

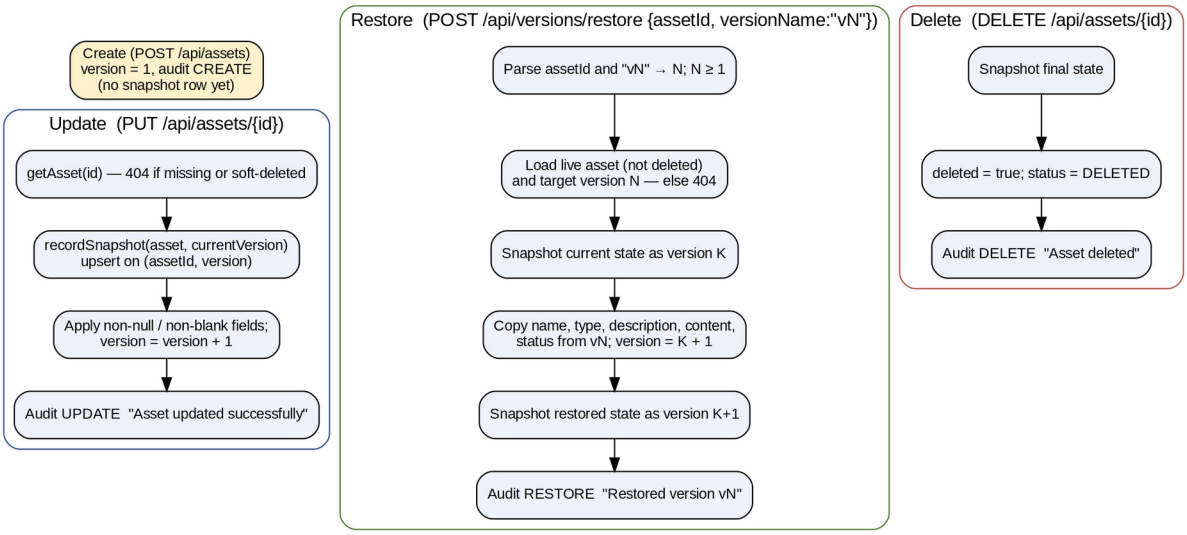


Figure 8.1: Asset versioning and restore workflow

The version API returns each snapshot as `{versionName, log}`, where `log.afterSnapshot` carries the captured field values. The frontend version page uses these snapshots to render a side-by-side diff (`diff-viewer.tsx`) and to trigger a restore after confirmation. The restore request body is `{"assetId": "3", "versionName": "v2"}`; the controller parses the numeric part of the version name and rejects malformed values with 400.

8.4 Audit Logging Module

Audit events are produced as a side effect of service methods, never by a client call (Figure 8.2). `ProtectedAssetService.createAuditLog` and `AssetVersionService.restoreVersion` construct an `AuditLog` with the actor returned by `currentActor()` — the authenticated user's e-mail, or the literal `SYSTEM` when there is no user principal — and save it in the same transaction as the change.

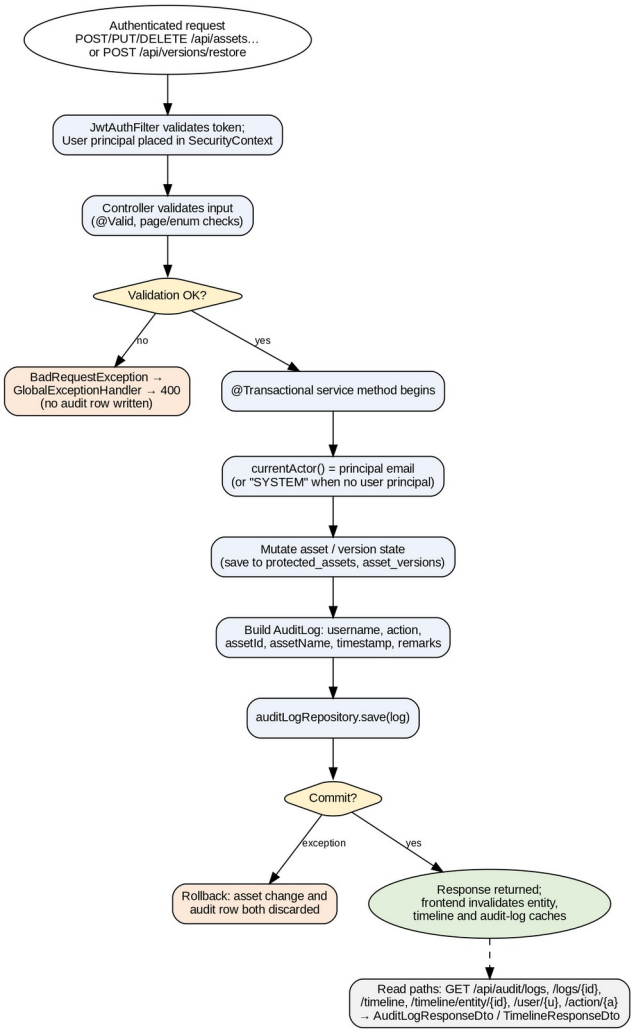


Figure 8.2: Audit-log write and read workflow

Table 8.1: Audit events produced by the system

Action	Produced by	Remarks text
CREATE	ProtectedAssetService.createAsset	Asset created
UPDATE	ProtectedAssetService.updateAsset	Asset updated successfully
DELETE	ProtectedAssetService.deleteAsset	Asset deleted
RESTORE	AssetVersionService.restoreVersion	Restored version v<N>

`AuditLogService` provides newest-first listing. Unfiltered requests use database pagination; requests with an `operation` (exact, case-insensitive) or `search` filter (substring over asset name, actor, action, remarks and asset id) are filtered in memory over the sorted list and then sliced, so `total` always reflects the filtered set. The timeline endpoint additionally accepts `assetId`. The response DTOs intentionally return empty `hash`, `previousHash`, `ipAddress` and `userAgent`, empty `fieldChanges`, a constant `severity = info` and `status = verified`, because the backend has no mechanism to populate them; the frontend no longer displays these fields.

8.5 Dashboard Module

`DashboardController` exposes five read endpoints backed by `DashboardService`: summary totals (assets, versions, audit logs, users), counts by audit action, counts of non-deleted assets by type, version snapshots per day, and the ten most recent audit events. The frontend renders these with stat cards, recharts charts and an activity feed, and provides a manual refresh button.

8.6 AI Assistant Module

8.6.1 Overview

The assistant (`backend/.../assistant`) is a read-only, retrieval-augmented question-answering service. It is **optional**: providers are always constructed so that the application boots, but report `isConfigured() == false` without a key, in which case chat returns HTTP 503 with code `assistant_not_configured`. It never writes to the core tables. Its components are document loaders, a chunker, embedding providers, a vector store, a retriever, a prompt builder, an LLM client, a tool catalogue and a citation formatter, orchestrated by `AssistantService`.

8.6.2 Document Ingestion

Two `DocumentLoader` implementations supply only explicitly approved, non-secret documents; neither scans arbitrary directories. `ClasspathDocumentLoader` loads `classpath*:assistant/knowledge/*.md` — eight curated files bundled with the backend (api-reference, assets, audit-and-timeline, authentication-and-security, dashboard, limitations, product-overview, versions-and-restore). `FileSystemDocumentLoader` loads the names in `assistant.indexing.documents` (default `PROTOTYPE_AUDIT.md, README-ENVIRONMENT.md`), resolving each against an optional workspace, the working directory and its parent, and skipping missing files with a log message. Each document becomes a `LoadedDocument(sourceId, title, path, content)`, where the title is the first `#` heading.

8.6.3 Chunking

`DocumentChunker` (defaults: 1000 characters, 150 overlap; minimum size 200, overlap capped at half the size) is deterministic so that unchanged content yields identical chunks. It walks the Markdown line by line, maintaining a heading stack; on each heading the accumulated body is flushed as chunks tagged with the section path (e.g. `5. Testing > 5.2 Frontend`). Body text is split into paragraphs, oversized paragraphs are hard-split near a space boundary, and paragraphs are packed into chunks; when a chunk is closed, a tail of the previous chunk is carried forward as overlap, but only as much as fits so that no chunk exceeds the maximum. Blank chunks are discarded.

8.6.4 Embedding Providers

Table 8.2: Embedding provider implementations

Provider	Endpoint / method	Default model & dimension	Notes
GeminiEmbeddingProvider	POST {base}/models/{model}:embedContent, key in x-goog-api-key	gemini-embedding-001, 768	model() returns 'model@dims' so a dimension change forces re-embedding
OpenAiEmbeddingProvider	POST {base}/embeddings, Bearer key	text-embedding-3-small, 1536	Any OpenAI-compatible base URL
LocalHashEmbeddingProvider	In-process	local-hash-bow-v1, 512	Signed bag-of-words hashing, L2-normalised; lexical only, for tests/offline

`AssistantConfig` selects providers at startup. If `ASSISTANT_LLM_PROVIDER` is blank, Gemini is chosen when `GEMINI_API_KEY` is present, otherwise OpenAI-compatible HTTP. The embedding provider follows the chat provider unless overridden, so a Gemini key is never sent to an OpenAI endpoint. All HTTP calls use Spring `RestClient` with configurable connect/read timeouts (default 120 s). Providers never return a fabricated vector: failures raise `AssistantProviderException`, and only the failure class or HTTP status is logged.

8.6.5 Vector Storage and Indexing

`VectorStore` is a narrow interface (find all, find by source, save, delete, count) with a default `JpaVectorStore` over `assistant_knowledge_chunks`; tests substitute an in-memory implementation. Vectors are serialised by `EmbeddingCodec` as JSON float arrays. `KnowledgeIndexer.indexAll()` (Figure 8.3) loads all documents, chunks them, and for each chunk computes a SHA-256 hash; if the stored row for `(sourceId, chunkIndex)` has the same hash, the same embedding model and a non-empty vector, the chunk is skipped, otherwise it is embedded and upserted. Rows whose index is beyond the new chunk count are deleted. The result reports embedded, skipped and removed counts. Indexing runs via `POST /api/assistant/reindex` (only if `ASSISTANT_INDEXING_ALLOW_API=true`, else 403) or at startup if `ASSISTANT_INDEX_ON_STARTUP=true`; startup failures are logged with an `ASSISTANT_DIAG` marker and never prevent the API from serving.

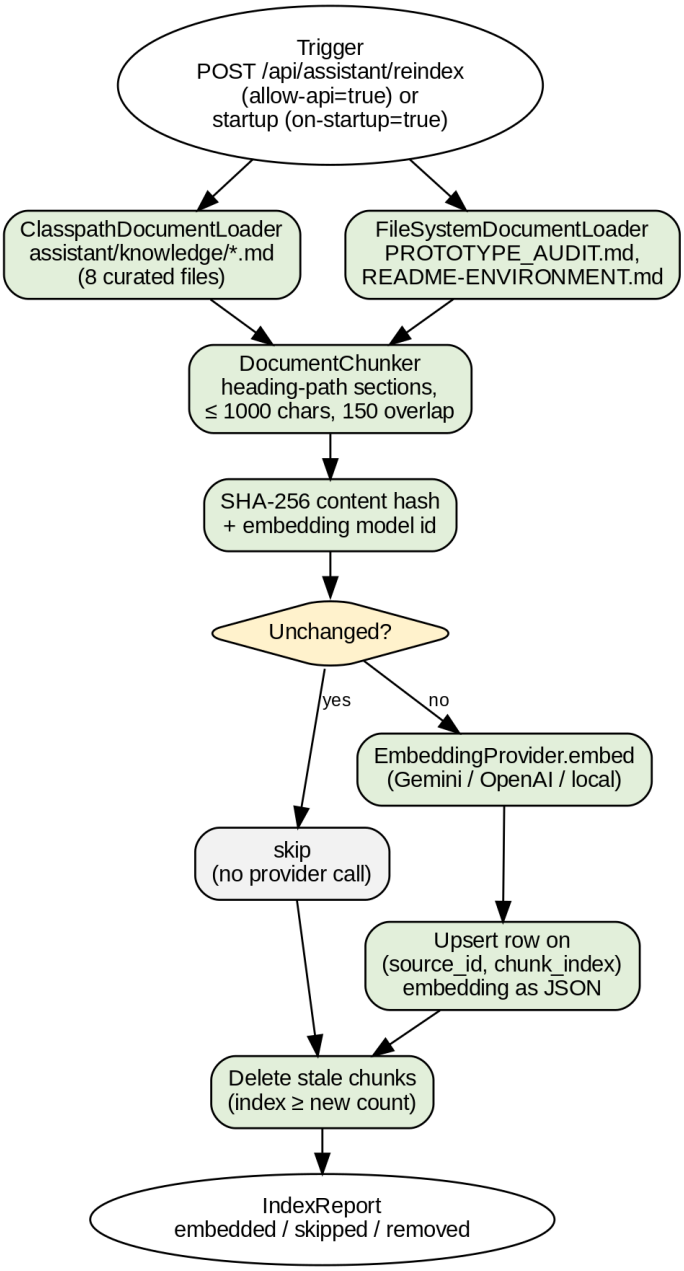


Figure 8.3: Knowledge ingestion and indexing pipeline

8.6.6 Retrieval

`Retriever.retrieve(query)` returns nothing for a blank query or empty index. Otherwise it loads up to `max-chunks` (5000) rows, embeds the query with the same provider, decodes each stored vector, computes cosine similarity (`EmbeddingCodec.cosine`, which returns 0 for empty, zero or mismatched-length vectors), keeps scores ≥ `min-score` (0.12), sorts descending and returns the top-k (6). Cosine similarity between query vector q and chunk vector d is computed as $q \cdot d / (\|q\| \|d\|)$. Figure 8.4 shows how retrieval fits into a complete assistant turn.

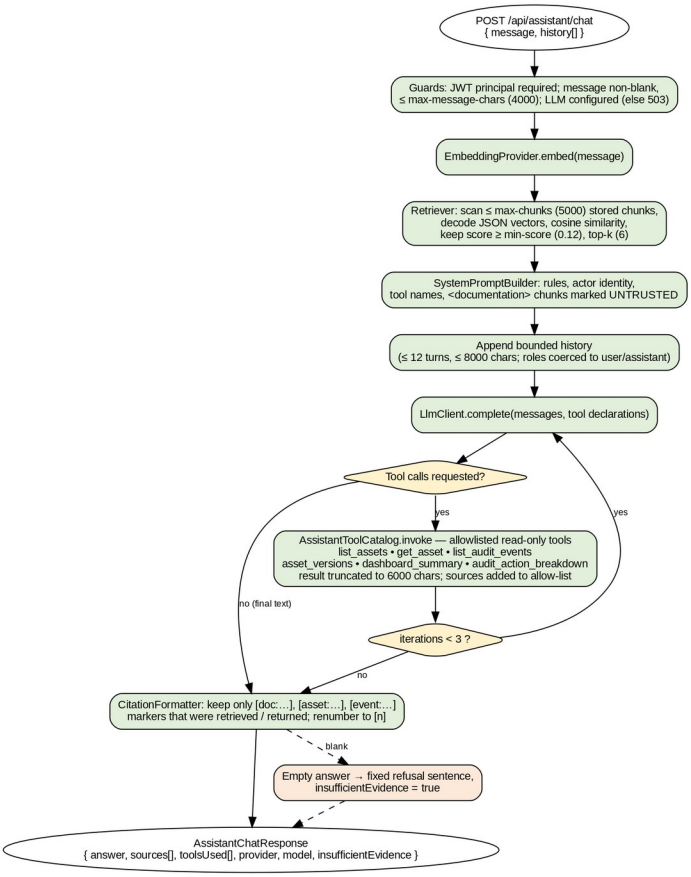


Figure 8.4: AI assistant retrieval and answer-generation workflow

8.6.7 Prompt Construction and Injection Guard

`SystemPromptBuilder` produces the system instruction. It states the acting identity (the authenticated e-mail and role — never a user id from the message), seven rules (read-only; ground every statement in documentation or a tool result, otherwise reply with the fixed sentence *"I couldn't find enough verified information to answer that."*; always use tools for current data; cite only with `[doc:<sourceId>#<chunkIndex>]`, `[asset:<id>]` or `[event:<id>]`; never claim IP capture, field diffs, hash chains or anomaly detection; label interpretation; answer in Markdown), and a prompt-injection guard declaring all documentation and tool output UNTRUSTED DATA. Retrieved chunks are wrapped in `<chunk id=… title=… section=…>` elements inside `<documentation>`, and any `chunk`, `document` or `documentation` tags inside the content are stripped so a document cannot close the wrapper.

8.6.8 Read-only Tool Catalogue

Table 8.3: Assistant tools (AssistantToolCatalog)

Tool	Parameters (validated)	Backed by	Cited as
list_assets	search, department, status∈{connected,syncing,disconnected,error}, risk∈{low,medium,high}, page, pageSize 1–50	ProtectedAssetService.getAllAssets	asset
get_asset	assetId ≥ 1 (required)	ProtectedAssetService.getAsset	asset
list_audit_events	assetId, operation∈{CREATE,UPDATE,DELETE,RESTO	AuditLogService.getTimeline	event

Tool	Parameters (validated)	Backed by	Cited as
	RE}, search, from/to ISO dates (from ≤ to), limit 1–100		
asset_versions	assetId ≥ 1 (required), limit 1–100	AssetVersionService.getVersions	asset
dashboard_summary	—	DashboardService.getSummary	—
audit_action_breakdown	—	DashboardService.getAuditActionCounts	—

There is no generic query or SQL tool. An unknown tool name or invalid JSON arguments produce an error result rather than an exception; every result is a compact, bounded JSON payload (truncated to 6000 characters) and the asset/event identifiers it contains are registered as allowed citation sources. At most three model/tool iterations are performed per question.

8.6.9 Citation Generation and Validation

`CitationFormatter.format(answer, allowed)` scans the model output with the pattern `\[(doc|asset|event):([^\]\s]+)]`. Each marker whose key is in the allowed map — documentation chunks retrieved for this question, or records returned by tools — is replaced by a sequential `[n]` and its `AnswerSource(type, id, title, section, href)` is appended to the sources list; repeated markers reuse the same number. Markers that do not resolve are **removed** and counted as dropped. If the cleaned answer is empty, the fixed refusal sentence is returned with `insufficientEvidence = true`. On the frontend, `citations.ts` maps `[n]` to its source, leaves out-of-range markers as plain text, and `safeHref` rejects `javascript:` and `data:` links.

8.6.10 Conversation Bounds and Error Handling

Messages are limited to 4000 characters; client-supplied history is truncated to the most recent 12 turns and 8000 characters, and roles other than `assistant` are coerced to `user`, so a client cannot inject a system message. `AssistantExceptionHandler` converts not-configured and provider errors into HTTP 503 with a generic message and `data = {code, reference}`, where the reference is a random 8-character correlation id that also appears in the server log line `Assistant 503 code=… ref=… cause=<ExceptionType>`. Provider messages are never logged because they may embed URLs or key fragments.

8.7 Frontend Modules

Table 8.4: Frontend pages

Route	Purpose
/auth/login, /auth/register	Sign-in and (if enabled) sign-up forms with zod validation
/dashboard	Totals, charts (actions, asset types, version growth) and recent activity
/entities	Protected assets table with search, filters, pagination, create/edit dialog and delete
/entities/versions	Version history for one asset, snapshot viewer, diff comparison and restore
/audit-logs	Paginated audit log table with operation/search filters and detail drawer
/timelines	Chronological audit timeline, optionally scoped to an asset
/assistant, /settings	Full-page assistant chat (suggested questions, Markdown answers, sources); profile and theme preferences

CHAPTER 9
SYSTEM IMPLEMENTATION

9.1 Development Environment

The backend is built with the Gradle wrapper on Java 21 and run with `./gradlew bootRun` (a PowerShell helper `start-local.ps1` is also provided). The frontend is run with `npm run dev`, and offers `lint`, `typecheck`, `test` and `build` scripts. Lombok reduces entity and DTO boilerplate (`@Builder`, `@Getter`, `@Setter`, `@Data`). This chapter presents representative excerpts of the implementation; excerpts are shortened but otherwise reproduce the repository code.

9.2 Backend Implementation Highlights

9.2.1 Fail-fast JWT key validation

```
@PostConstruct
void init() {
    if (jwtSecret == null || jwtSecret.isBlank())
        throw new IllegalStateException("JWT_SECRET environment variable is not
set. ...");
    byte[] keyBytes;
    try { keyBytes = Base64.getDecoder().decode(jwtSecret.trim()); }
    catch (IllegalArgumentException e) { throw new
IllegalStateException("JWT_SECRET is not valid Base64. ...", e); }
    if (keyBytes.length < MIN_SECRET_BYTES)          // 32 bytes = 256 bits
        throw new IllegalStateException("JWT_SECRET decodes to only " +
keyBytes.length + " bytes ...");
    this.signingKey = Keys.hmacShaKeyFor(keyBytes);
}
```

Listing 9.1: JwtService.init() (abridged)

9.2.2 Per-request authentication with database-sourced role

```
String token = authHeader.substring(7);
if (!jwtService.isTokenValid(token)) { filterChain.doFilter(request, response);
return; }
String email = jwtService.extractEmail(token);
if (email != null && SecurityContextHolder.getContext().getAuthentication() ==
null) {
    User user = userRepository.findByEmail(email).orElse(null);
    if (user != null) {
        String role = user.getEffectiveRole();          // authoritative role from
DB
        var auth = new UsernamePasswordAuthenticationToken(
                user, null, List.of(new SimpleGrantedAuthority("ROLE_" + role)));
        SecurityContextHolder.getContext().setAuthentication(auth);
    }
}
filterChain.doFilter(request, response);
```

Listing 9.2: JwtAuthFilter.doFilterInternal() (abridged)

9.2.3 Transactional update with snapshot and audit

```
@Transactional
public AssetResponseDto updateAsset(Long id, UpdateAssetRequest request) {
```

```
    ProtectedAsset asset = getAsset(id);          // 404 if missing or soft-
deleted
    String actor = currentActor();                // principal e-mail or "SYSTEM"
    saveVersion(asset, actor);                    // 1. snapshot current state
    updateCurrentAsset(asset, request, actor);    // 2. apply fields, version + 1
    createAuditLog(asset, "UPDATE", "Asset updated successfully", actor);   // 3.
audit
    return AssetResponseDto.from(asset);
}
```

Listing 9.3: ProtectedAssetService.updateAsset()

9.2.4 History-preserving restore

```
recordSnapshot(asset, asset.getVersion(), actor, now);        // preserve current
state
asset.setAssetName(target.getAssetName());
asset.setAssetType(target.getAssetType());
asset.setDescription(target.getDescription());
asset.setContent(target.getContent());
asset.setStatus(target.getStatus());
asset.setUpdatedBy(actor); asset.setUpdatedAt(now);
asset.setVersion((asset.getVersion() == null ? 1 : asset.getVersion()) + 1);
protectedAssetRepository.save(asset);
AssetVersion restored = recordSnapshot(asset, asset.getVersion(), actor, now);
auditLogRepository.save(AuditLog.builder().username(actor).action("RESTORE")
        .assetId(asset.getId()).assetName(asset.getAssetName()).timestamp(now)
        .remarks("Restored version v" + versionNumber).build());
```

Listing 9.4: core of AssetVersionService.restoreVersion()

9.2.5 Cosine similarity and citation validation

```
public static double cosine(float[] a, float[] b) {
    if (a == null || b == null || a.length == 0 || a.length != b.length) return
0.0;
    double dot = 0, normA = 0, normB = 0;
    for (int i = 0; i < a.length; i++) { dot += a[i]*b[i]; normA += a[i]*a[i];
normB += b[i]*b[i]; }
    if (normA == 0.0 || normB == 0.0) return 0.0;
    return dot / (Math.sqrt(normA) * Math.sqrt(normB));
}
// CitationFormatter: keep only verified markers, renumber to [n]
while (matcher.find()) {
    String key = matcher.group(1) + ":" + matcher.group(2);
    AnswerSource source = allowed.get(key);
    if (source == null) { dropped++; matcher.appendReplacement(out, ""); continue;
}
    Integer n = indexByKey.computeIfAbsent(key, k -> { sources.add(source); return
sources.size(); });
    matcher.appendReplacement(out, Matcher.quoteReplacement("[" + n + "]"));
}
```

Listing 9.5: EmbeddingCodec.cosine() and the CitationFormatter loop (abridged)

9.3 Frontend Implementation Highlights

`httpClient` wraps `fetch`: it builds the URL from `env.apiBaseUrl` (default `/api`), serialises query parameters, attaches the bearer token, parses JSON, maps 204 to `undefined`, and throws `HttpClientError(message, statusCode)` for non-2xx responses (including non-JSON bodies). Service modules expose only endpoints that exist on the backend; the frontend test `mock-free-contract.test.ts` guards against reintroduction of the deleted mock API layer. Pagination helpers translate one-based UI

pages to the backend’s zero-based pages, and list services forward only supported filter keys.

```
const token = typeof window !== 'undefined' ?
localStorage.getItem('sentinel_audit_token') : null;
const response = await fetch(buildUrl(path, params), {
  headers: { 'Content-Type': 'application/json', Accept: 'application/json',
             ...(token ? { Authorization: `Bearer ${token}` } : {}), ...headers },
  body: body !== undefined ? JSON.stringify(body) : undefined, ...rest });
if (!response.ok) {
  let errorPayload: ApiError = { message: 'Request failed' };
  try { errorPayload = (await response.json()) as ApiError; } catch { /* non-JSON
*/ }
  throw new HttpClientError(errorPayload, response.status);
}
if (response.status === 204) return undefined as T;
return (await response.json()) as T;
```

Listing 9.6: request() in services/http-client.ts (abridged)

After create, update, delete or restore, `invalidateAssetSideEffects` invalidates the entity, timeline and audit-log query caches (but not unrelated ones such as the dashboard), so the next view refetches fresh data. The assistant UI (`assistant-chat.tsx`, `floating-assistant.tsx`) sends the message and prior turns, renders Markdown answers with numbered citations linked to in-app pages, shows a “not configured” banner from `/api/assistant/status`, and maps 503/401/5xx/network errors to user-friendly messages (`chat-errors.ts`).

9.4 Error Handling

Controllers contain no try/catch blocks; typed exceptions propagate to `GlobalExceptionHandler`, which logs the detail server-side and returns a stable, generic `ApiResponse {success:false, message, data:null}`. This prevents stack traces, SQL messages or internal identifiers from reaching clients.

Table 9.1: Exception-to-status mapping

Exception	HTTP status	Client message
BadRequestException, MethodArgumentNotValidException, HttpMessageNotReadable, type mismatch, missing parameter, IllegalArgumentException	400	The request was invalid.
UnauthorizedException, AuthenticationException, missing/invalid JWT (entry point)	401	Authentication is required.
ForbiddenException, AccessDeniedException	403	You do not have permission to perform this action.
RegistrationDisabledException	403	Registration is disabled on this deployment.
NotFoundException, NoResourceFoundException	404	The requested resource was not found.
HttpRequestMethodNotSupportedException	405	The HTTP method is not supported for this endpoint.
ConflictException (duplicate e-mail)	409	The request conflicts with the current state.
AssistantNotConfiguredException / AssistantProviderException	503	Not configured / temporarily unavailable, plus {code, reference}
Any other Exception	500	An unexpected error occurred.

9.5 Build and Deployment

```
# Backend (requires JWT_SECRET and DATABASE_* in the environment)
cd backend && ./gradlew bootRun            # development
./gradlew test                              # automated tests (H2, no external
services)
docker build -t sentinelaudit-backend .     # multi-stage image, runs java -jar
app.jar on :8080

# Frontend
cd frontend && npm install && npm run dev   # http://localhost:3000, /api proxied
to :8080
npm run lint && npm run typecheck && npm test && npm run build
```

Listing 9.7: build and run commands

The Dockerfile builds the boot JAR inside an `eclipse-temurin:21` image and copies it into a smaller `eclipse-temurin:21-jre` runtime image. Secrets are injected at run time as environment variables; the root `.gitignore` excludes `.env` files, build outputs and logs. Enabling the assistant in a deployment requires a provider key and a one-time index build (Appendix C).

CHAPTER 10
API DOCUMENTATION

10.1 Conventions

- Base path `/api`; JSON request and response bodies (`Content-Type: application/json`).
- Authentication: `Authorization: Bearer <accessToken>` on every endpoint except those marked Public.
- Pagination: `page` is **zero-based**; `pageSize` 1–200 (default 50). Paginated responses have the shape `{data[], total, page, pageSize, totalPages}`.
- Errors: `{success:false, message, data}` with the status codes in Table 9.2; assistant 503 responses carry `data.code` and `data.reference`.
- Example values below are illustrative; field names and types are taken from the DTO classes.

10.2 Endpoint Summary

Table 10.1: REST endpoint summary

Method	Path	Auth	Description
GET	/api/health	Public	Liveness text
GET	/api/auth/config	Public	{registrationEnabled}
POST	/api/auth/register	Public	Create account → 201 AuthResponse (403 if disabled, 409 duplicate)
POST	/api/auth/login	Public	Authenticate → 200 AuthResponse (401 on failure)
GET	/api/auth/me	JWT	Current UserResponse
POST	/api/auth/logout	JWT	204 (stateless no-op)
POST	/api/assets	JWT	Create asset → 201 AssetResponseDto
GET	/api/assets	JWT	List with page, pageSize, search, department, status, risk
GET	/api/assets/{id}	JWT	Get asset (404 if deleted/missing)
PUT	/api/assets/{id}	JWT	Partial update → 200 AssetResponseDto
DELETE	/api/assets/{id}	JWT	Soft delete → 204
GET	/api/versions/asset/{assetId}	JWT	Paginated version history (newest first)
GET	/api/versions/asset/{assetId}/{version}	JWT	Single version snapshot
POST	/api/versions/restore	JWT	Restore {assetId, versionName}
GET	/api/audit/logs	JWT	Paginated logs; search, operation
GET	/api/audit/logs/{id}	JWT	Single audit log
GET	/api/audit/timeline	JWT	Paginated timeline; assetId, search, operation
GET	/api/audit/timeline/entity/{entityId}	JWT	Full timeline for one asset (list)
GET	/api/audit/user/{username}	JWT	Logs by actor (legacy lookup)
GET	/api/audit/action/{action}	JWT	Logs by action (legacy lookup)
GET	/api/dashboard/summary	JWT	Totals
GET	/api/dashboard/audit-actions	JWT	[{action, count}]
GET	/api/dashboard/assets-by-type	JWT	[{assetType, count}] (non-deleted)
GET	/api/dashboard/version-growth	JWT	[{date, count}]
GET	/api/dashboard/recent-activities	JWT	Ten most recent AuditLog entities
GET	/api/assistant/status	JWT	AssistantStatusResponse (no secrets)
POST	/api/assistant/chat	JWT	Grounded answer (400 invalid, 503 unavailable)
POST	/api/assistant/reindex	JWT	Re-index knowledge (403 unless enabled)

10.3 Authentication Endpoints

```
POST /api/auth/register            (POST /api/auth/login takes {email, password})
{ "fullName": "Asha Rao", "email": "asha@example.org", "password": "*********" }

201 Created
{ "accessToken": "<JWT>", "tokenType": "Bearer", "expiresIn": 86400,
  "user": { "id": 1, "name": "Asha Rao", "email": "asha@example.org",
            "role": "USER", "avatarUrl": null } }
```

Listing 10.1: register request and AuthResponse

10.4 Asset Endpoints

`CreateAssetRequest` requires `assetName` and `assetType` and accepts `description`, `content`, `status`, `department`, `ownerName`, `ownerEmail`, `risk`, `sourceApplication`. `UpdateAssetRequest` has the same fields, all optional. The response adapts the entity to the frontend's `Entity` type:

```
GET /api/assets?page=0&pageSize=20&search=payroll&risk=high
200 OK
{ "data": [ {
    "id": "3", "name": "database", "displayName": "Payroll DB", "category":
"Database",
    "description": "HR payroll store", "sourceApplication": "SentinelAudit",
    "recordCount": 0, "changeCount": 2, "lastChangedAt": "2026-10-20T10:15:02",
    "createdAt": "2026-10-19T09:00:00", "schema": [], "department": "Finance",
    "owner": { "id": "u-3", "name": "Finance Lead", "email": "lead@example.org",
"role": "USER", "avatarUrl": null },
    "status": "connected", "version": "v3", "risk": "high" } ],
  "total": 1, "page": 0, "pageSize": 20, "totalPages": 1 }
```

Listing 10.2: asset list response (illustrative values)

Unsupported query parameters (e.g. `sort`), `page < 0`, `pageSize` outside 1–200, filter strings longer than 200 characters, or invalid `status`/`risk` values return 400.

10.5 Version Endpoints

```
POST /api/versions/restore
{ "assetId": "3", "versionName": "v1" }

200 OK
{ "success": true, "message": "Version v1 restored successfully",
  "restoredVersion": { "versionName": "v4",
    "log": { "id": "11", "entityId": "3", "entityName": "Payroll DB", "operation":
"",
             "actor": { "name": "asha@example.org", ... },
             "reason": "Captured state for version 4",
             "afterSnapshot": { "assetName": "Payroll DB", "assetType":
"Database",
                                "description": "...", "content": "...", "status":
"ACTIVE" },
             "hash": "", "previousHash": "", "createdAt": "2026-10-
20T10:20:00" } } }
```

Listing 10.3: restore request and response (abridged)

10.6 Audit and Timeline Endpoints

`AuditLogResponseDto` fields: `id`, `entityId`, `entityName`, `operation`, `severity` (`"info"`), `status` (`"verified"`), `actor{id,name,email,role,avatarUrl}`, `sourceApplication`, `ipAddress` (`""`), `userAgent` (`""`), `reason`, `fieldChanges` (`[]`), `beforeSnapshot` (`{}`), `afterSnapshot` (`{}`), `hash` (`""`), `previousHash` (`""`), `createdAt`. `TimelineResponseDto` fields: `id`, `auditLogId`, `entityId`, `entityName`, `operation`, `actor`, `summary` (the remark, or "<action> <asset name>"), `fieldCount` (`0`), `hash`, `previousHash`, `createdAt`.

10.7 Dashboard Endpoints

```
GET /api/dashboard/summary        → { "totalAssets": 12, "totalVersions": 30,
"totalAuditLogs": 45, "totalUsers": 3 }
GET /api/dashboard/audit-actions  → [ { "action": "CREATE", "count": 12 },
{ "action": "UPDATE", "count": 25 } ]
GET /api/dashboard/version-growth → [ { "date": "2026-10-19", "count": 8 } ]
```

Listing 10.4: dashboard responses (illustrative values)

10.8 Assistant Endpoints

```
POST /api/assistant/chat
{ "message": "Who last changed asset 3?",
  "history": [ { "role": "user", "content": "..." }, { "role": "assistant",
"content": "..." } ] }

200 OK
{ "answer": "Asset 3 was last updated by asha@example.org [1].",
  "sources": [ { "type": "event", "id": "41", "title": "...", "section": null,
"href": "..." } ],
  "toolsUsed": ["list_audit_events"], "provider": "gemini", "model": "gemini-3.8-
flash",
  "insufficientEvidence": false }

503 Service Unavailable
{ "success": false, "message": "The assistant is not configured on this server.",
  "data": { "code": "assistant_not_configured", "reference": "1f2e3d4c" } }
```

Listing 10.5: assistant chat exchange (illustrative values)

`GET /api/assistant/status` returns `configured`, `llmProvider`, `llmModel`, `embeddingProvider`, `embeddingModel`, `indexedChunks`, `indexedSources[]`, `indexingEnabled`, `indexingOnStartup`. `POST /api/assistant/reindex` returns `sources[]`, `embeddedChunks`, `skippedChunks`, `removedChunks`, `embeddingModel`.

CHAPTER 11
TESTING AND VALIDATION

11.1 Testing Strategy

Testing combines (i) **unit tests** of pure components (chunker, citation formatter, retriever, prompt builder, provider configuration, pagination helpers), (ii) **integration tests** that start the Spring application context with an in-memory H2 database and exercise endpoints through the real security filter chain with real JWTs, (iii) **contract tests** on the frontend that assert the exact requests the client sends and how responses and errors are interpreted, and (iv) **static checks** (TypeScript type-checking and ESLint). External model providers are never called by the test suite: the test profile sets the embedding provider to `local` and leaves LLM keys empty, and provider clients are exercised against stubbed HTTP responses.

11.2 Backend Test Suites

Table 11.1: Backend test classes and number of @Test methods (from source)

Test class	Tests	Focus
BackendApplicationTests	1	Application context loads
controller.UnauthenticatedSecurityRegressionTest	9	Anonymous access to assets, audit, dashboard, versions, /me → 401; health public; bad/malformed login
controller.AuthenticatedCoreWorkflowIntegrationTest	3	Full lifecycle with real JWT, actor attribution, history preservation; failed mutation writes no audit row
controller.AuditTimelineRegressionTest	9	Zero-based paging, newest-first order, backend-applied filters, page-size limits
controller.AssetFilteringRegressionTest	5	Search beyond first page, case-insensitive department, status aliases, totals, 400 on invalid params
controller.AssetPaginationRegressionTest	4	Created asset visible and readable; zero-based paging
controller.RegistrationConfigurationTest	5	Registration enabled/disabled behaviour; login unaffected
dto.TimelineResponseDtoTest	3	Timeline DTO mapping
assistant.AssistantServiceTest	8	Grounded answers, citation dropping, refusal, provider failure not masked, tool loop, length limits
assistant.AssistantControllerTest	6	Actor from principal, DTO handling, error mapping
assistant.AssistantEndpointSecurityTest	4	401 without JWT, 503 unconfigured, 400 blank message, status without secrets
assistant.AssistantGeminiRoundTripTest	3	Gemini request/response translation round trip
assistant.provider.GeminiLlmClientTest	13	Message/tool translation, blocked responses, error handling
assistant.provider.GeminiEmbeddingProviderTest	6	Request shape, dimensions, error paths
assistant.provider.AssistantConfigTest	8	Provider auto-selection and key resolution
assistant.tools.AssistantToolCatalogTest	6	Allow-list, argument validation, date filtering, unknown tool
assistant.knowledge.DocumentChunkerTest	4	Section metadata, overlap, size bound, no empty chunks
assistant.knowledge.KnowledgeIndexerTest	3	Idempotent re-index, stale-chunk removal, failure surfaces
assistant.retrieval.RetrieverTest	3	Cosine ranking, top-k and min-score, metadata
assistant.citation.CitationFormatterTest	4	Renumbering, dropping unverifiable markers
assistant.prompt.SystemPromptBuilderTest	3	Grounding rules, untrusted-content guard, refusal sentence
Total	**110**	

11.3 Frontend Test Suites

Table 11.2: Frontend Vitest suites (counts as reported by the test run)

Test file	Tests	Focus
src/mock-free-contract.test.ts	24	Deleted mock modules stay deleted; proxy and /api default; no workspace header; honest naming
src/lib/pagination.test.ts	9	One-based UI ⇄ zero-based API page translation
src/features/assistant/lib/chat-errors.test.ts	8	User-facing messages for 503, 401, 5xx, 4xx, network and unknown errors
src/services/list-contract.test.ts	7	Only supported filters forwarded; timeline endpoint and assetId; no invented hashes
src/features/assistant/lib/citations.test.ts	6	[n] → source mapping, out-of-range markers, safe links
src/services/assistant.test.ts	5	Chat/status wire contract; no client-supplied identity; 503/400 errors
src/services/http-client.test.ts	5	Error mapping, 401 preserved, non-JSON errors, 204, bearer token
src/features/assistant/lib/suggested-questions.test.ts	3	Suggested prompts contain no pre-baked answers
src/hooks/use-entities.test.ts	1	Cache invalidation after asset mutations
Total	**68**	

11.4 Representative Test Cases

Table 11.3: Representative test cases and expected behaviour

ID	Test (class › method)	Input / action	Expected result
TC1	UnauthenticatedSecurityRegressionTest › assetsEndpointRejectsAnonymousRequests	GET /api/assets without token	401 JSON
TC2	UnauthenticatedSecurityRegressionTest › healthEndpointStaysPublic	GET /api/health	200
TC3	UnauthenticatedSecurityRegressionTest › loginWithBadCredentialsReturns…	Wrong password	401, generic message
TC4	AuthenticatedCoreWorkflowIntegrationTest › fullLifecycleWritesAuditTrail…	Register → create → update → restore → update → delete	Snapshots and events correct; actor = token e-mail; history readable after delete
TC5	AuthenticatedCoreWorkflowIntegrationTest › failedMutationCreatesNoAuditEvent	Invalid PUT	Error; no new audit row
TC6	AssetFilteringRegressionTest › searchFindsMatchesBeyondTheFirstPage	130 seeded rows; search	Match found; total reflects full set
TC7	AssetFilteringRegressionTest › unsupportedAndInvalidParametersAreRejected…	sort=…, status=bogus	400
TC8	RegistrationConfigurationTest › registerIsRejectedWith…	REGISTRATION_ENABLED =false	403; login still works
TC9	AuditTimelineRegressionTest › timelineOperationFilterIsAppliedByTheBackend	operation=UPDATE	Only UPDATE events
TC10	AssistantServiceTest › dropsCitationsThatDoNotResolveToRetrievedContent	Model cites unknown chunk	Marker removed
TC11	AssistantServiceTest › propagatesProviderFailureInsteadOfFabricatingAnAnswer	Provider throws	Exception → 503; no answer
TC12	AssistantEndpointSecurityTest › assistantEndpointsRequireAuthentication	Chat without JWT	401
TC13	citations.test.ts › rejects javascript:, data: and empty values	Unsafe href	Link not rendered
TC14	assistant.test.ts › never sends a client-supplied user identity	Chat request body	Only message and history

11.5 Test Execution Results

The suites were executed for this report on a clean copy of the repository at commit `5e57b1e` (Ubuntu 22.04, OpenJDK 21, Node.js 22). Results are reported exactly as observed:

Table 11.4: Observed test and build results

Check	Command	Result
Backend tests	./gradlew test	BUILD SUCCESSFUL — 110 tests in 22 result files, 0 failures, 0 errors, 0 skipped
Frontend unit/contract tests	npx vitest run	9 test files passed, 68 tests passed, 0 failed
Frontend type check	npm run typecheck (tsc --noEmit)	Passed (no errors reported)
Frontend lint	npm run lint	Passed — “No ESLint warnings or errors”
Frontend production build	npm run build	Passed — compiled successfully; 13/13 static pages generated

Two reproducibility observations were made while executing the suites. (1) `npm ci` failed because `package-lock.json` is out of sync with `package.json` (missing `@types/node@26.6.6` and `undici-types@8.9.0` entries), so dependencies were installed with `npm install` instead. (2) With the dependencies resolved by `npm install` (Vitest 3.2.7 with Vite 7.3.7), `vitest.config.ts` could not be loaded as CommonJS (`ERR_REQUIRE_ESM`); the tests were therefore run with an identical copy of the configuration saved as `vitest.config.mts` in the temporary test copy. No test or application file was changed, and the repository itself was not modified.

Backend dependencies were downloaded through a Maven Central mirror configured in a local Gradle init script after the default repository returned HTTP 429 (rate limiting); this affects only dependency download, not the build or test configuration.

For comparison, the developer’s engineering record (`PROTOTYPE_AUDIT.md` §11.5, dated 2026-10-20) records 110 backend tests with 0 failures, 68 passing frontend tests, clean lint and type checks and a successful production build (13/13 pages).

11.6 Validation Not Performed

- No live model provider was called; answer quality and retrieval quality with real semantic embeddings are not verified by automated tests (the local embedding provider is lexical).
- No load, performance or penetration testing was carried out, so no performance figures are claimed.
- No browser-level end-to-end (click-through) test suite exists in the repository.
- No user-acceptance study or field evaluation is recorded in the repository.

CHAPTER 12
RESULTS AND DISCUSSION

12.1 Delivered Functionality

The implemented system meets the functional requirements FR1–FR10 of Chapter 4, subject to the restrictions listed in Table 2.1 and Chapter 13. Table 12.1 traces each requirement to its implementation and to automated evidence.

Table 12.1: Requirement traceability

Req.	Implementation	Automated evidence
FR1	AuthController, UserService, JwtService	RegistrationConfigurationTest; lifecycle test registers and uses a real token
FR2	SecurityConfig, JwtAuthFilter, JSON entry point	UnauthenticatedSecurityRegressionTest (9 tests)
FR3	ProtectedAssetController / Service	AuthenticatedCoreWorkflowIntegrationTest, AssetPaginationRegressionTest
FR4	JPA Specification, parameter validation	AssetFilteringRegressionTest; list-contract.test.ts
FR5	AssetVersionService.recordSnapshot, version endpoints	Lifecycle test (pre-update snapshot, history after delete)
FR6	AssetVersionService.restoreVersion	Lifecycle test (content restored, history preserved, no duplicate versions)
FR7	createAuditLog / RESTORE event in same transaction	Lifecycle test (actor equals authenticated e-mail; failed PUT writes no event)
FR8	AuditLogService, AuditLogController	AuditTimelineRegressionTest (9 tests)
FR9	DashboardController / Service	Context and security tests (endpoints protected); no dedicated aggregate test
FR10	assistant package	AssistantService/Controller/Endpoint tests, retriever, chunker, indexer, citation, prompt and provider tests

12.2 User Interface

The following screenshots were captured from the application running locally against a PostgreSQL database populated through the application’s own API during report preparation (sample data, not production data). The assistant was not configured with a provider key, so its page shows the “not configured” state.

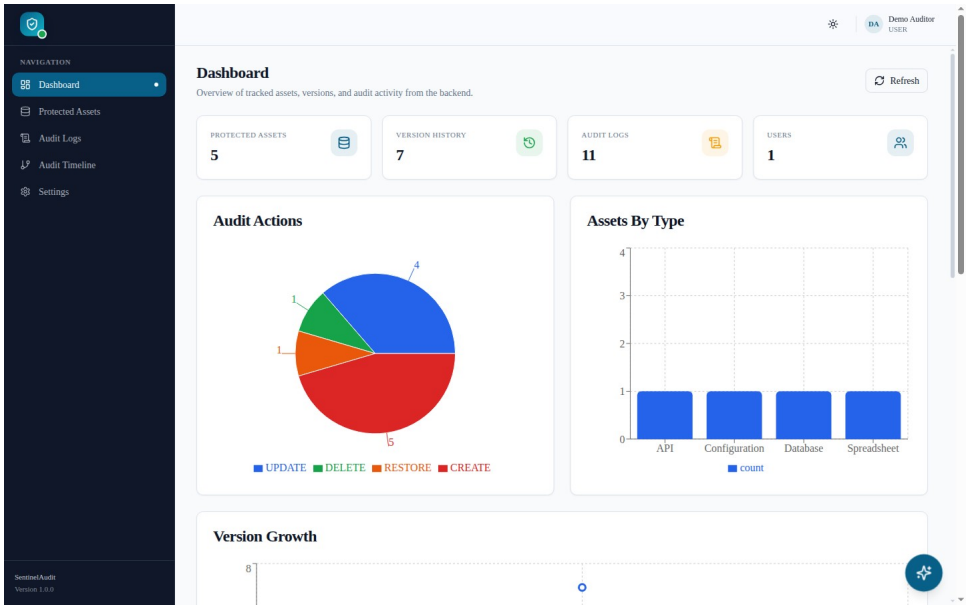


Figure 12.1: Dashboard with totals, charts and recent activity

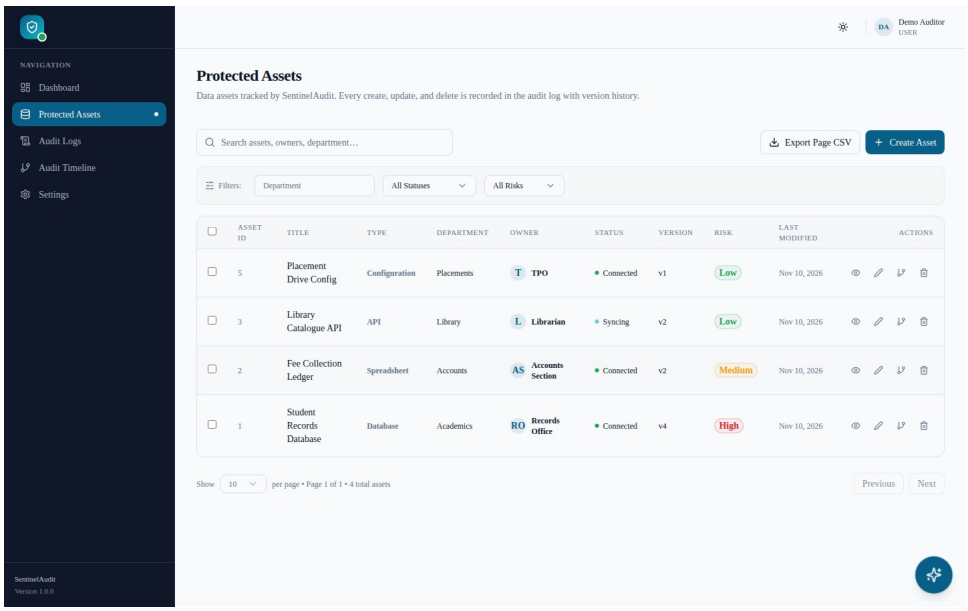


Figure 12.2: Protected assets list with search, filters and pagination

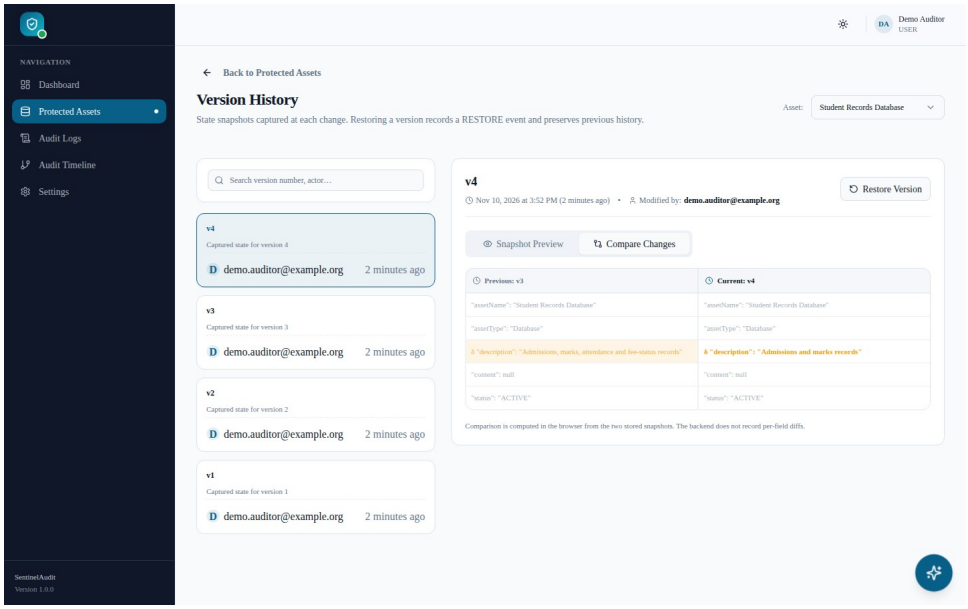


Figure 12.3: Version history of an asset

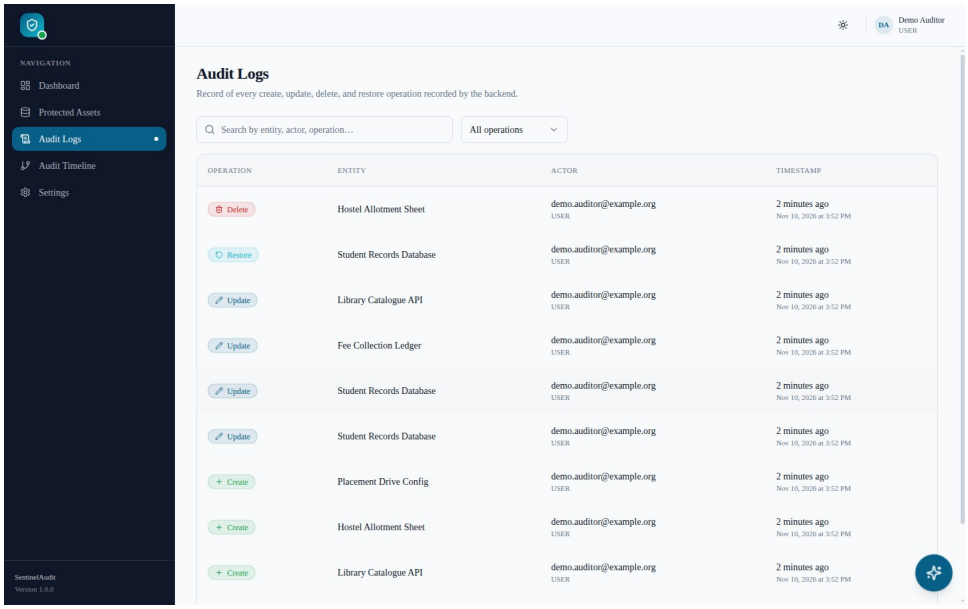


Figure 12.4: Audit log table

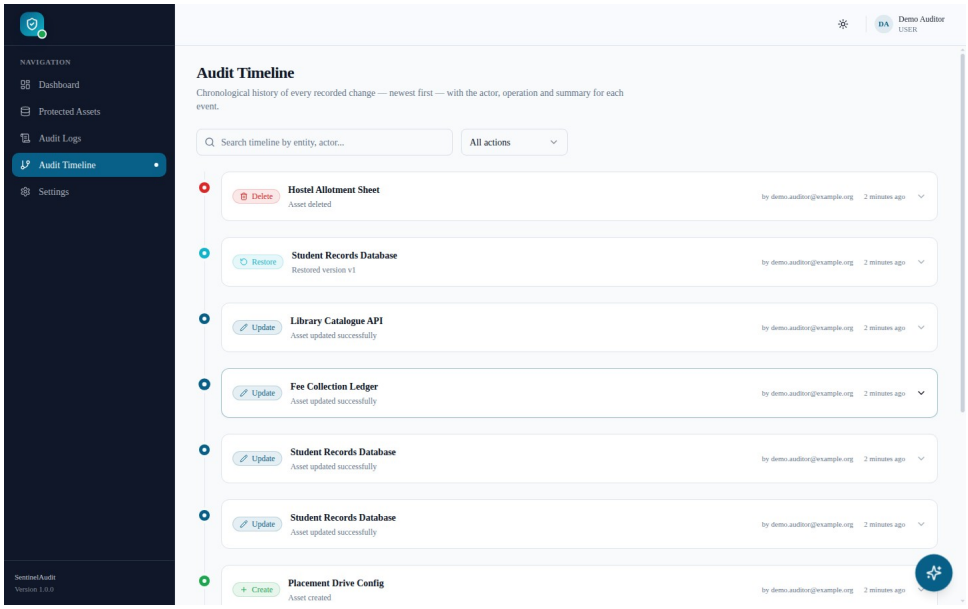


Figure 12.5: Audit timeline

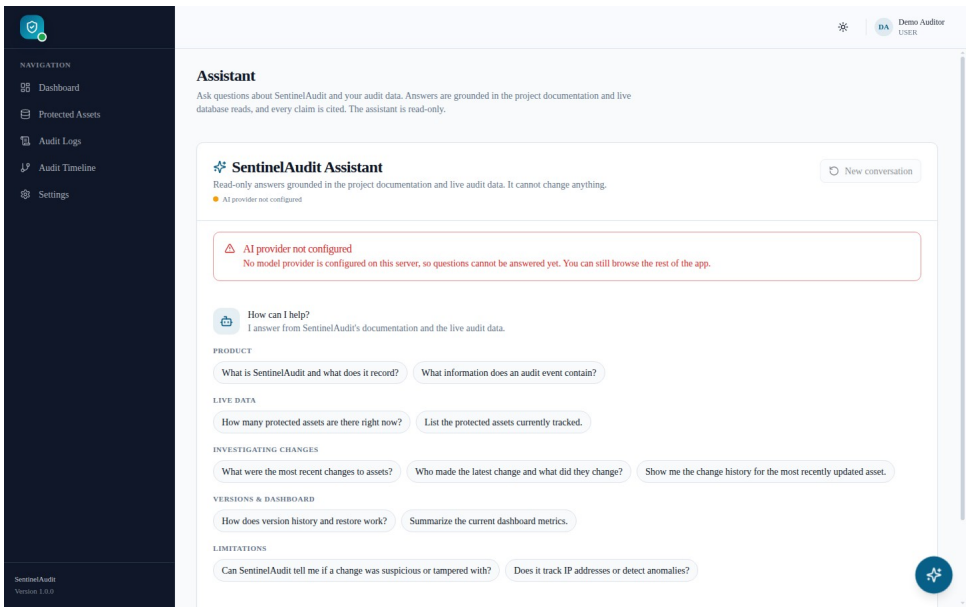


Figure 12.6: Assistant page in the not-configured state

12.3 Discussion

12.3.1 Atomic, server-attributed audit trail

Deriving the actor from the security context rather than from request data, and writing the audit row in the same transaction as the change, are the two decisions that make the audit trail trustworthy at the application level. The integration test that performs a failing `PUT` and asserts that no audit row appears demonstrates the second property directly. The audit trail is not, however, tamper-evident against a database administrator; Chapter 14 proposes hash chaining for that purpose.

12.3.2 Non-destructive versioning

Restore is modelled as a forward operation that produces a new version rather than as a rollback. As a consequence the version history is monotonic and complete: a user can restore v1, inspect the result, and still return to the pre-restore state, which itself was snapshotted. The upsert on `(assetId, version)` fixed a defect in which two consecutive restores could create duplicate version numbers.

12.3.3 Honest data contracts

The frontend was originally built against a richer mock contract (hash chains, IP addresses, field changes, severities). Rather than fabricating such values, the backend returns empty or constant values and the UI was changed to stop displaying them; a frontend contract test asserts that empty hash values pass through as empty. This trades visual richness for accuracy, which is the correct trade-off for an accountability tool.

12.3.4 Grounded assistant design

The assistant's safety rests on mechanisms that do not depend on model behaviour: identity comes from the JWT; tools are a fixed allow-list of read-only service calls with validated parameters; tool output and documents are framed as untrusted data; history roles are coerced; and citations are validated after generation so that a fabricated reference cannot reach the user as a link. Provider failures surface as HTTP 503 with a correlation id instead of a fallback answer. The in-process vector store is a deliberate simplicity choice suitable for the small curated corpus (eight bundled documents plus two repository documents), not a scalable design.

12.3.5 Observations from executing the build

Re-running the project from a clean checkout surfaced two environment-level observations relevant to maintainers: `npm ci` fails because `package-lock.json` is not in sync with `package.json`, and, after `npm install`, the default `vitest.config.ts` could not be loaded as a CommonJS config with the resolved Vite 7 (Section 11.5). Neither affects application behaviour, but both affect reproducibility and are listed among the limitations. In addition, during automated screenshot capture a full page load of a protected deep link (for example `/entities`) by a logged-in user ended on `/dashboard` rather than the requested page, whereas client-side navigation through the sidebar worked as expected. This was observed in an automated browser session only and has not been root-caused; a likely area to inspect is the interaction between session rehydration in `AuthProvider` and the redirect in `ProtectedRoute`.

CHAPTER 13
LIMITATIONS

The following limitations are observable in the source code or are documented by the developer in `PROTOTYPE_AUDIT.md` and `README-ENVIRONMENT.md`.

1. **No tamper evidence.** Audit rows are not hash-chained or signed; `hash` and `previousHash` are always empty. A database administrator could alter or delete rows without detection.
2. **Limited audit context.** IP address, user agent and server-side field-level diffs are not captured; `severity` is always `info` and `status` always `verified`.
3. **No anomaly detection or risk scoring.** The feature was deliberately deferred; the `risk` field is a user-entered label.
4. **Single-tenant data model.** Every authenticated user can read and modify every asset and read every audit log; there is no per-user or per-organisation scoping.
5. **Roles not enforced.** All self-registered users receive `USER`; no endpoint is restricted by role and there is no admin provisioning flow. Consequently the re-index endpoint, when enabled, is available to any authenticated user.
6. **Stateless logout.** `POST /api/auth/logout` does not revoke the token; it remains valid until expiry (24 h by default). There is no refresh-token flow.
7. **Token storage in localStorage.** The access token is readable by JavaScript on the page, which increases the impact of any cross-site scripting flaw.
8. **No database-level integrity for versions and references.** `(asset_id, version)` uniqueness and asset references are enforced only by the service layer; pre-existing duplicate version rows are not cleaned up.
9. **Schema managed by `ddl-auto=update`.** There is no versioned migration tool, so destructive or data-transforming schema changes are not handled.
10. **In-memory filtering for filtered audit queries.** Filtered audit-log and timeline requests load all audit rows and filter in the JVM, which will not scale to very large logs.
11. **Assistant scalability and scope.** Retrieval scans up to 5000 chunks in process (no pgvector or ANN index); answers are not streamed; conversations are not persisted; live answer quality with real providers is not verified by automated tests.
12. **Data sharing with providers.** When the assistant is enabled, questions, document excerpts and tool results (which may include asset and audit content) are sent to the external provider.
13. **Asset `content` not editable in the UI.** The field is accepted by the API but deliberately not collected by the form; `recordCount` is always 0 and is not displayed.

14. **Deep-link reload behaviour.** A full page load of a protected route by a logged-in user was observed to land on the dashboard instead of the requested page (Section 12.3.5).
15. **Reproducibility issues.** `package-lock.json` is out of sync with `package.json`, and the Vitest configuration requires an ESM-compatible load with current Vite releases.
16. **No CI pipeline, end-to-end browser tests, load tests or security audit** are present in the repository.

CHAPTER 14
CONCLUSION AND FUTURE ENHANCEMENTS

14.1 Conclusion

SentinelAudit demonstrates that an attributable audit trail, non-destructive versioning and a grounded AI assistant can be delivered with a conventional, open-source web stack and modest infrastructure. The Spring Boot backend enforces stateless JWT authentication with fail-fast key validation, writes each asset change, its prior-state snapshot and its audit event atomically, and translates every error into a uniform, non-leaking response. The Next.js frontend consumes only real endpoints and avoids displaying data the backend cannot substantiate. The assistant implements a complete RAG pipeline — curated ingestion, deterministic chunking, pluggable embeddings, idempotent indexing, cosine retrieval, bounded read-only tool use and post-hoc citation validation — and degrades to an explicit 503 rather than an invented answer. The test suites executed for this report (Chapter 11) support these claims at the level of automated verification, while Chapter 13 records the guarantees the system does not yet provide.

14.2 Future Enhancements

1. **Tamper-evident audit log:** chain each audit row to its predecessor with a SHA-256 hash (and optionally sign periodic checkpoints), and expose a verification endpoint.
2. **Richer audit context:** capture client IP and user agent, and store server-computed field-level diffs alongside snapshots.
3. **Role-based access control and multi-tenancy:** introduce administrator and auditor roles with `@PreAuthorize` rules, organisation scoping of assets and logs, and an admin-only re-index endpoint.
4. **Session security:** short-lived access tokens with rotating refresh tokens, server-side revocation on logout, and HttpOnly cookie storage with CSRF protection.
5. **Database hardening:** Flyway/Liquibase migrations, foreign keys and a unique constraint on `(asset_id, version)` after de-duplication, and database-level pagination for filtered audit queries.
6. **Scalable retrieval:** optional `pgvector` column with an approximate-nearest-neighbour index, hybrid lexical + semantic ranking, streaming responses and evaluation datasets for answer quality.
7. **Operational maturity:** CI pipeline running tests, lint and builds on every push; containerised frontend and docker-compose for local deployment; health/readiness probes and structured logging.
8. **Anomaly detection and accessibility:** rule-based alerts (e.g. bulk deletions, off-hours activity) clearly labelled as heuristics, and an accessible, localised UI for community organisations working in regional languages.

REFERENCES

[1] VMware / Broadcom, “Spring Boot Reference Documentation,” https://docs.spring.io/spring-boot/

[2] VMware / Broadcom, “Spring Security Reference,” https://docs.spring.io/spring-security/reference/

[3] VMware / Broadcom, “Spring Data JPA Reference Documentation,” https://docs.spring.io/spring-data/jpa/reference/

[4] M. Jones, J. Bradley and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, IETF, May 2015.

[5] M. Jones, “JSON Web Algorithms (JWA),” RFC 7518, IETF, May 2015.

[6] JJWT project, “Java JWT: JSON Web Token for Java and Android,” https://github.com/jwtk/jjwt

[7] N. Provos and D. Mazières, “A Future-Adaptable Password Scheme,” in Proc. USENIX Annual Technical Conference, FREENIX Track, 1999.

[8] The PostgreSQL Global Development Group, “PostgreSQL Documentation,” https://www.postgresql.org/docs/

[9] Hibernate ORM, “Hibernate ORM User Guide,” https://hibernate.org/orm/documentation/

[10] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in Advances in Neural Information Processing Systems 33 (NeurIPS), 2020.

[11] Vercel, “Next.js Documentation,” https://nextjs.org/docs

[12] Meta Open Source, “React Documentation,” https://react.dev/

[13] TanStack, “TanStack Query Documentation,” https://tanstack.com/query/latest

[14] Google, “Gemini API Documentation — Text generation, function calling and embeddings,” https://ai.google.dev/gemini-api/docs

[15] OpenAI, “API Reference — Chat Completions and Embeddings,” https://platform.openai.com/docs/api-reference

[16] OWASP Foundation, “OWASP Top 10 for Large Language Model Applications,” https://owasp.org/www-project-top-10-for-large-language-model-applications/

[17] OWASP Foundation, “OWASP Top Ten Web Application Security Risks,” https://owasp.org/www-project-top-ten/

[18] K. Kent and M. Souppaya, “Guide to Computer Security Log Management,” NIST Special Publication 800-92, 2006.

[19] Docker Inc., “Dockerfile reference — multi-stage builds,” https://docs.docker.com/

[20] Vitest, “Vitest Documentation,” https://vitest.dev/

[21] SentinelAudit project repository, https://github.com/Rahulkagita/SENTINEL-AUDIT-final (commit 5e57b1e), including README-ENVIRONMENT.md and PROTOTYPE_AUDIT.md.

APPENDICES

Appendix A – Repository Structure

```
SENTINEL-AUDIT-final/
├── README-ENVIRONMENT.md          environment variables, assistant setup,
troubleshooting
├── PROTOTYPE_AUDIT.md             engineering record of changes, tests and
limitations
├── backend/                       Spring Boot service
│   ├── Dockerfile, build.gradle, settings.gradle, gradlew(.bat), start-local.ps1
│   └── src/main/java/com/sentinelaudit/backend/
│       ├── assistant/ {citation, dto, knowledge, prompt, provider, retrieval,
tools, vector}
│       ├── config/  controller/  dashboard/  dto/  entity/  exception/
│       ├── repository/  response/  security/  service/
│   └── src/main/resources/{application.properties, assistant/knowledge/*.md}
│   └── src/test/java/...          21 test classes (+ InMemoryVectorStore support)
└── frontend/                      Next.js application
    ├── next.config.js, netlify.toml, package.json, tailwind.config.ts,
vitest.config.ts
    └── src/{app, components, features, hooks, lib, providers, services, types}
```

Appendix B – Environment Template (no secrets)

```
# Backend — set in the process environment or a secrets manager; never commit
values
JWT_SECRET=<base64 value decoding to at least 32 bytes, e.g. from: openssl rand -
base64 64>
JWT_EXPIRATION=86400000
DATABASE_URL=jdbc:postgresql://<host>:5432/sentinelaudit
DATABASE_USERNAME=<db user>
DATABASE_PASSWORD=<db password>
CORS_ALLOWED_ORIGINS=https://<frontend-origin>
REGISTRATION_ENABLED=true
# Optional assistant (choose one provider)
GEMINI_API_KEY=<key>            # or ASSISTANT_LLM_API_KEY=<key> with
ASSISTANT_LLM_PROVIDER=openai
ASSISTANT_INDEXING_ALLOW_API=false
ASSISTANT_INDEX_ON_STARTUP=false

# Frontend
API_PROXY_TARGET=http://localhost:8080
# NEXT_PUBLIC_API_BASE_URL=https://<backend-origin>/api   (optional; never put
secrets in NEXT_PUBLIC_*)
```

Appendix C – Build, Run and Assistant Indexing Procedure

1. Create a PostgreSQL database named `sentinelaudit` (or point `DATABASE_URL` at an existing one).
2. Export the backend variables of Appendix B and run `cd backend && ./gradlew bootRun`; verify `GET http://localhost:8080/api/health`.
3. Run `cd frontend && npm install && npm run dev` and open `http://localhost:3000`; register or log in.
4. To enable the assistant, set a provider key and restart; then temporarily set `ASSISTANT_INDEXING_ALLOW_API=true`, call `POST /api/assistant/reindex` with a valid JWT, and set it back to `false`.

5. Verify with `GET /api/assistant/status`: `configured: true` and a non-zero `indexedChunks`.
6. For container deployment, build the backend image with `docker build -t sentinelaudit-backend backend/` and supply the same environment variables at run time.

Appendix D – Assistant Knowledge Sources

Table A.1: Documents indexed by the assistant

Source id	Location	Loader
api-reference	backend/src/main/resources/assistant/knowledge/	ClasspathDocumentLoader
assets	same	ClasspathDocumentLoader
audit-and-timeline	same	ClasspathDocumentLoader
authentication-and-security	same	ClasspathDocumentLoader
dashboard	same	ClasspathDocumentLoader
limitations	same	ClasspathDocumentLoader
product-overview	same	ClasspathDocumentLoader
versions-and-restore	same	ClasspathDocumentLoader
prototype_audit	repository root (PROTOTYPE_AUDIT.md)	FileSystemDocumentLoader
readme-environment	repository root (README-ENVIRONMENT.md)	FileSystemDocumentLoader

Appendix E – Repository Commit History

Table A.2: Commit history of the repository (git log)

Commit	Date	Message
2b9bd21	2026-10-18	Initial combined SentinelAudit frontend and backend
056c5c4	2026-10-19	fixed backend build
232f23e	2026-10-19	Fix asset tab and audit timeline
9817d36	2026-10-19	Remove mock data and streamline SentinelAudit
800c992	2026-10-19	added a chatbot assistant in the backend
5f0d3f9	2026-10-19	updated the chatbot
64fbf9c	2026-10-20	updated the environment variables
ff7cba1	2026-10-20	Prepare SentinelAudit for demo deployment
5e57b1e	2026-10-20	Add Docker deployment for backend

Appendix F – Community Service Activity Records

[To be completed by the student(s). Attach the officially certified Community Service Project records required by the college — for example, the activity log/diary, details of the community or organisation engaged, dates of interaction, attendance and coordinator signatures. These records are not part of the software repository and have intentionally not been generated in this document.]

Date	Activity	Place / Organisation	Coordinator signature